

E. Generative Modeling

E.1 Generative Modeling: Basic concepts

E.2 Generative Modeling: Variational models

E.3 Generative Modeling: Diffusion models I

Intuition from a simple physical system

E.4 Generative Modeling: Diffusion models II

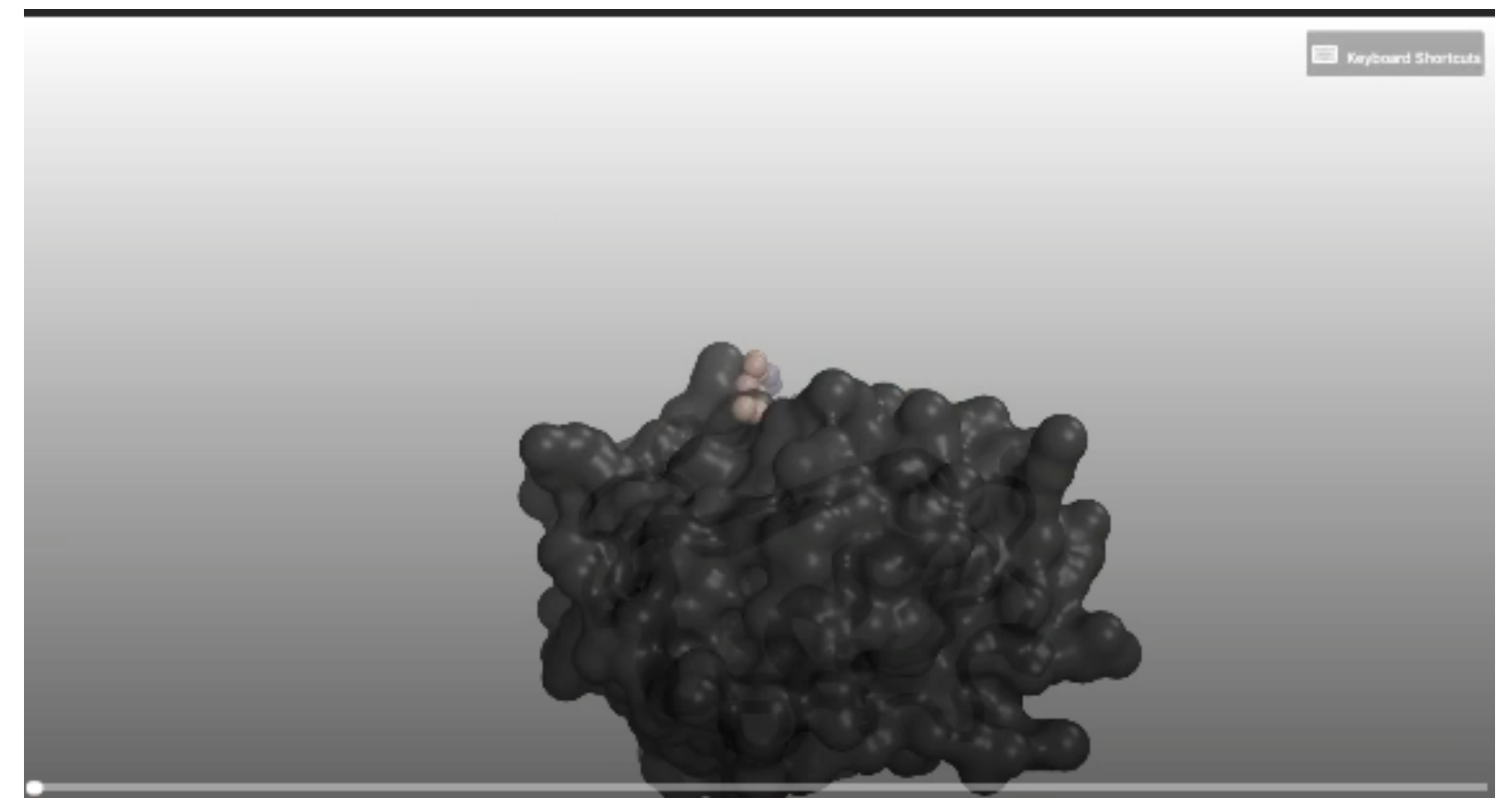
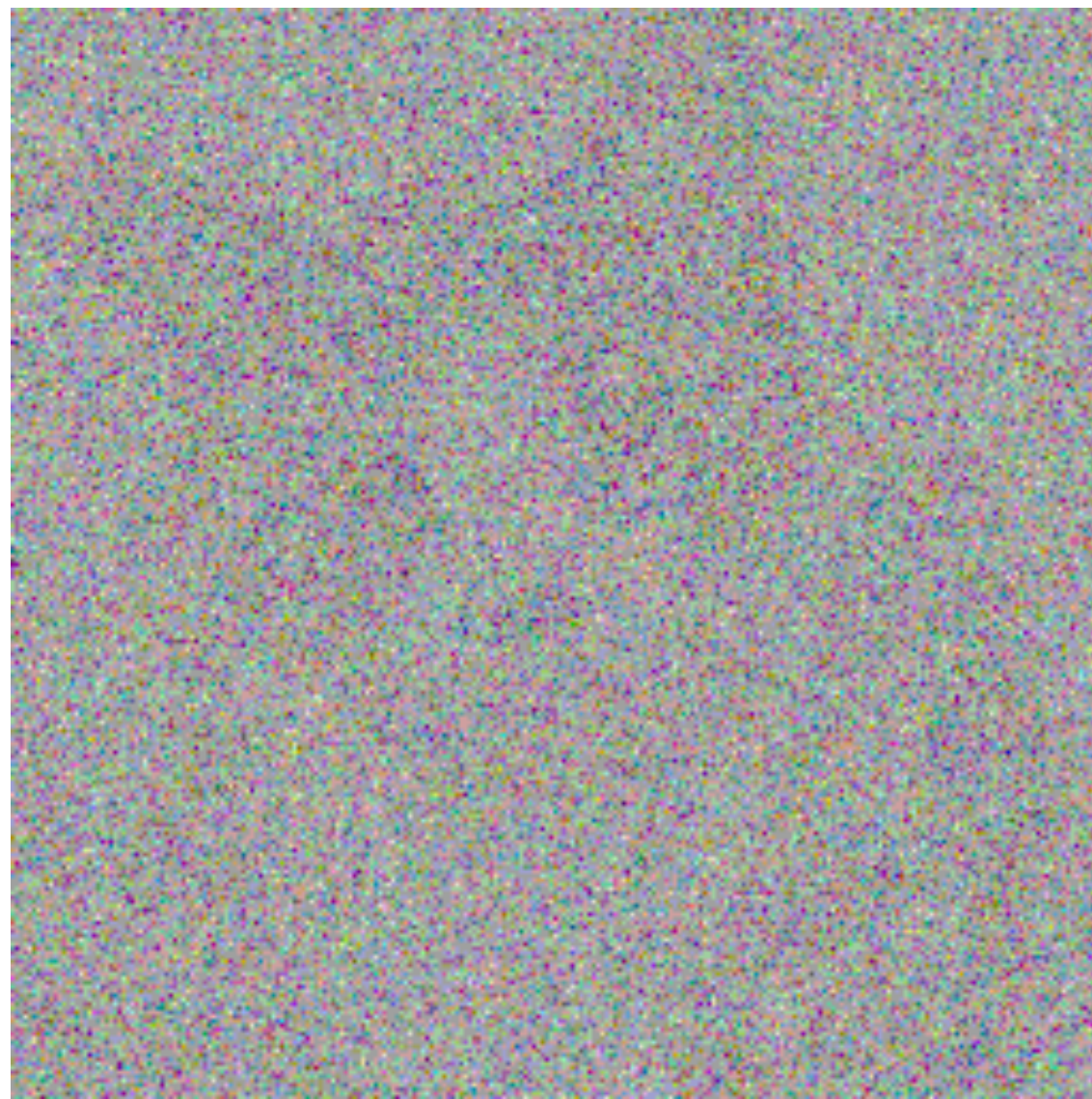
E.5 Generative Modeling: Structured (conditional) generation



Tiam Heydari, PhD
Postdoctoral Fellow, UBC

Diffusion Models

Diffusion now sits behind some of the biggest scientific wins in generative AI including the leading protein-design models.



But the way it's written up is a composition of seemingly-different methods.

REVERSE-TIME DIFFUSION EQUATION MODELS*

Brian D.O. ANDERSON**
Department of Electrical Engineering, The University of Newcastle, N.S.W. 2308, Australia

Received 13 November 1979
Revised 12 May 1980

Deep Unsupervised Learning using
Nonequilibrium Thermodynamics

Jascha Sohl-Dickstein
Stanford University
Eric A. Weiss
University of California, Berkeley
Niru Maheswaranathan
Stanford University
Surya Ganguli
Stanford University

JASCHA@STANFORD.EDU
EWEISS@BERKELEY.EDU
NTRUM@STANFORD.EDU
SGANGULI@STANFORD.EDU

Estimation of Non-Normalized Statistical Models
by Score Matching

Aapo Hyvärinen
*Helsinki Institute for Information Technology (HIIT)
Department of Computer Science
FIN-00014 University of Helsinki, Finland*

AAPO.HYVARINEN@HEL.SINKI.FI

Editor: Peter Dayan

Variational Diffusion Models

Diederik P. Kingma*, Tim Salimans*, Ben Poole, Jonathan Ho
Google Research

Denoising Diffusion Probabilistic Models

Jonathan Ho UC Berkeley jonathanno@berkeley.edu
Ajay Jain UC Berkeley ajayj@berkeley.edu
Pieter Abbeel UC Berkeley pabbeel@cs.berkeley.edu

Abstract

Improved Techniques for Training Score-Based
Generative Models

Yang Song
Computer Science Department
Stanford University
yangsong@cs.stanford.edu

Stefano Ermon
Computer Science Department
Stanford University
ermon@cs.stanford.edu

Sliced Score Matching: A Scalable Approach to
Density and Score Estimation

Yang Song* Stanford University
Sahaj Garg* Stanford University
Jiaxin Shi Tsinghua University
Stefano Ermon Stanford University

DPM-Solver: A Fast ODE Solver for Diffusion
Probabilistic Model Sampling in Around 10 Steps

Cheng Lu†, Yuhao Zhou†, Fan Bao†, Jianfei Chen†*, Chongxuan Li†, Jun Zhu†*
Dept. of Comp. Sci. & Tech., Institute for AI, BNRist Center, THBI Lab
†Tsinghua-Bosch Joint ML Center, Tsinghua University, Beijing, 100084 China
†Gaoling School of Artificial Intelligence, Renmin University of China
‡Beijing Key Laboratory of Big Data Management and Analysis Methods, Beijing, China
{lucheng,lc15,yuhaoz.cs}@gmail.com; bfi19@mails.tsinghua.edu.cn
chongxuanli@ruc.edu.cn; {jianfeic,dczsj}@tsinghua.edu.cn

SCORE-BASED GENERATIVE MODELING THROUGH
STOCHASTIC DIFFERENTIAL EQUATIONS

Yang Song* Stanford University yangsong@cs.stanford.edu
Jascha Sohl-Dickstein Google Brain jaschasd@google.com
Diederik P. Kingma Google Brain durk3@google.com

Abhishek Kumar Google Brain abhishek@google.com
Stefano Ermon Stanford University ermon@cs.stanford.edu
Ben Poole Google Brain pooleb@google.com

Section 1: Finding our path to diffusion through physical systems

Diffusion models can potentially be introduced from many directions:

probability

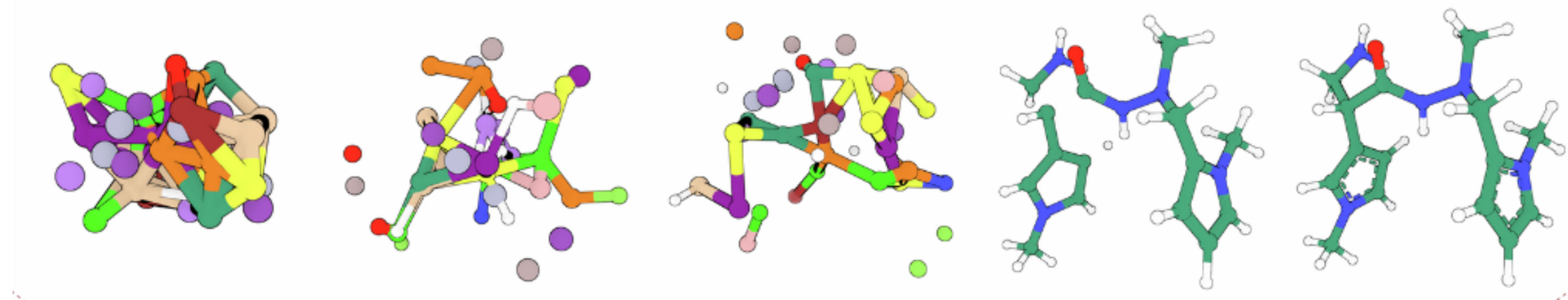
variational inference

score matching

stochastic processes

ODEs

....



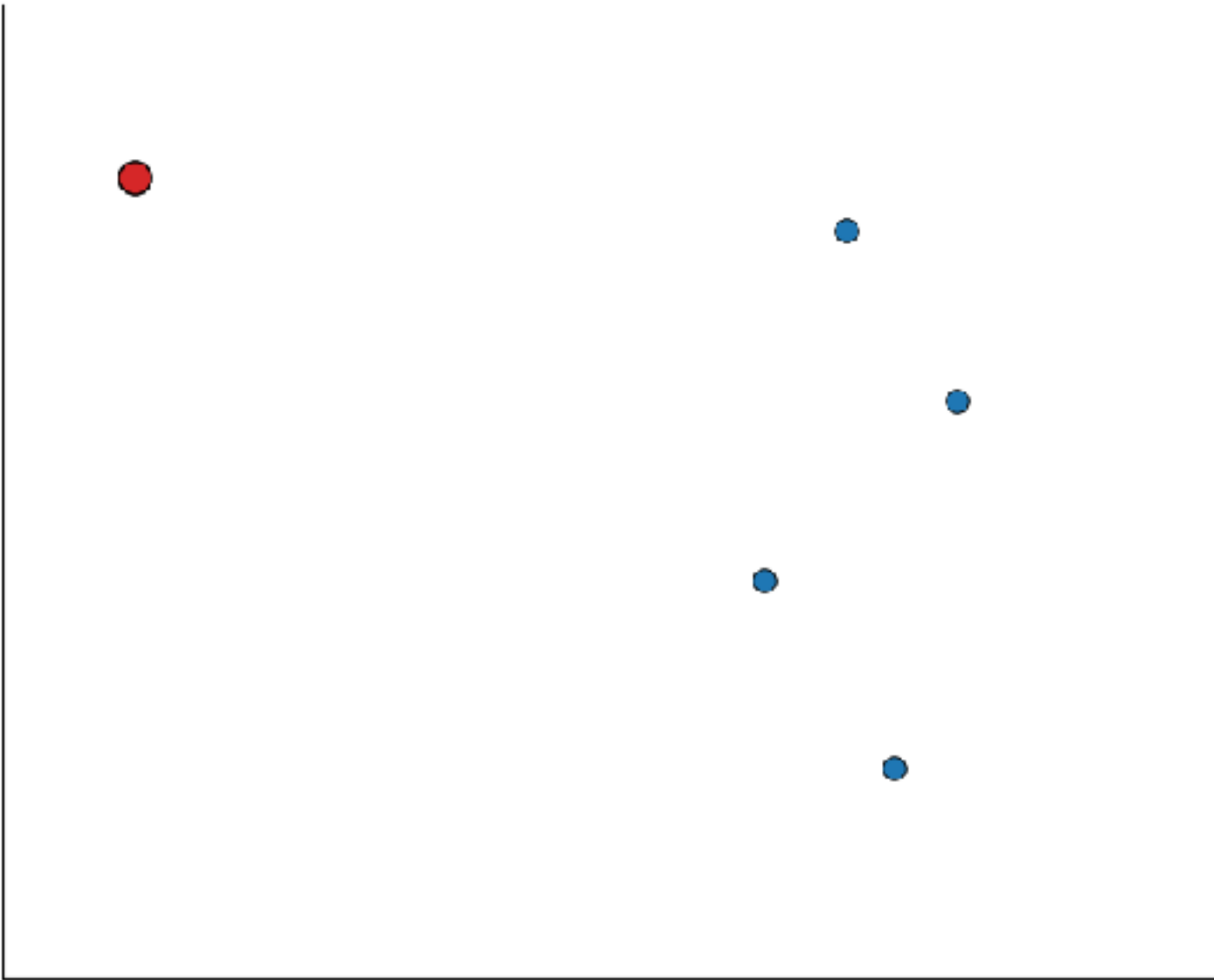
In this lecture, we will make build our own path based on some intuitions from simple physical systems:

Some charged particles → fields and forces → ODEs → SDEs → diffusion models

- We start with a more physical question: if particles are sitting in a field, how do they move?
- Once we understand that, diffusion models become much less mysterious. A diffusion model is mostly a way of learning the field that moves noisy particles back toward data.

Section 1| A charged particle in a physical system:The setup

Imagine we place several fixed **negative charges** in space .
Then we release **one positive charge** somewhere else.
In a viscous environment, the positive charge slowly drifts according to the pull it feels from the system.



Section 1| A charged particle in a physical system: **The potential scalar field**

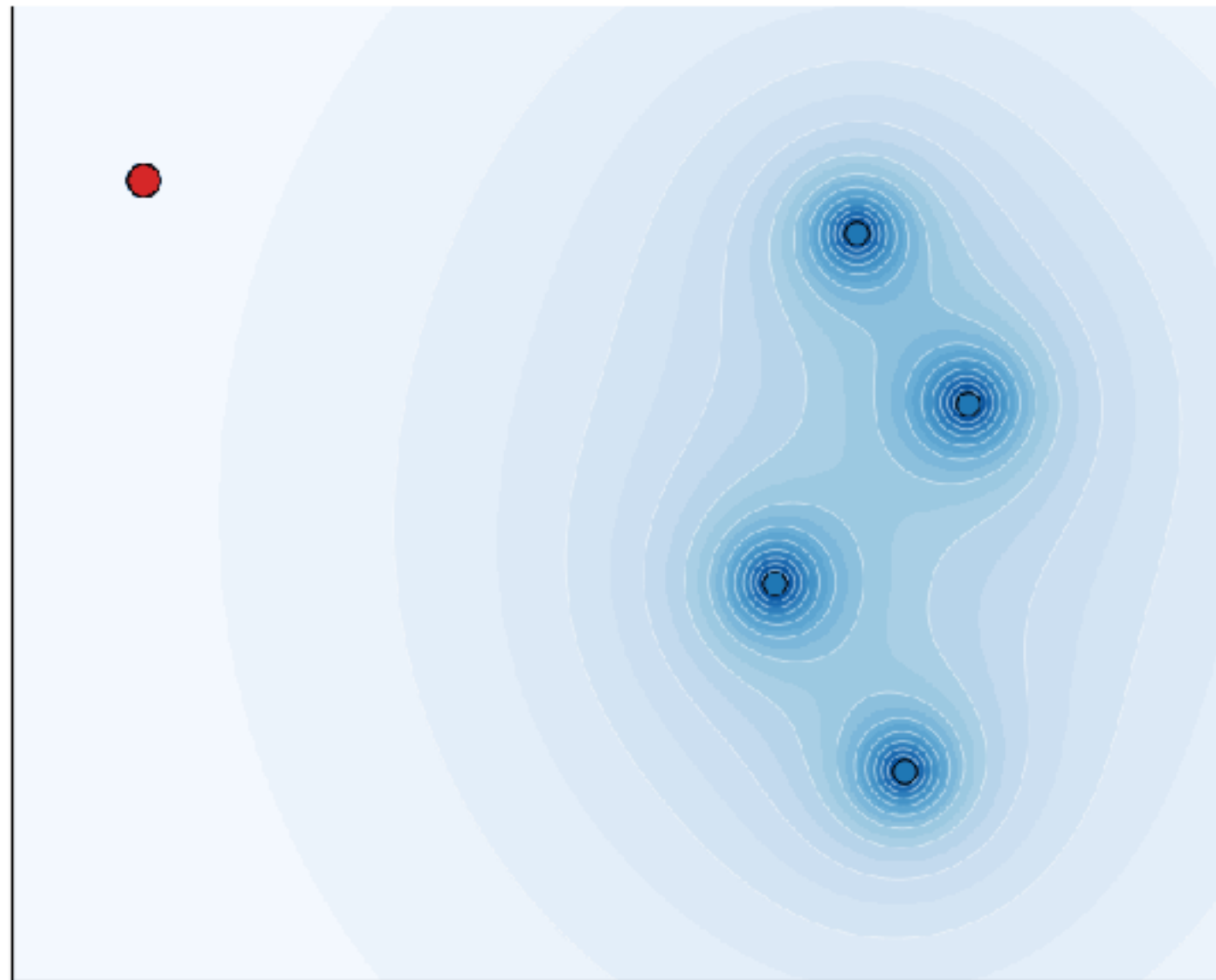
Why does it the particle move?

The fixed charges create a potential landscape in space.

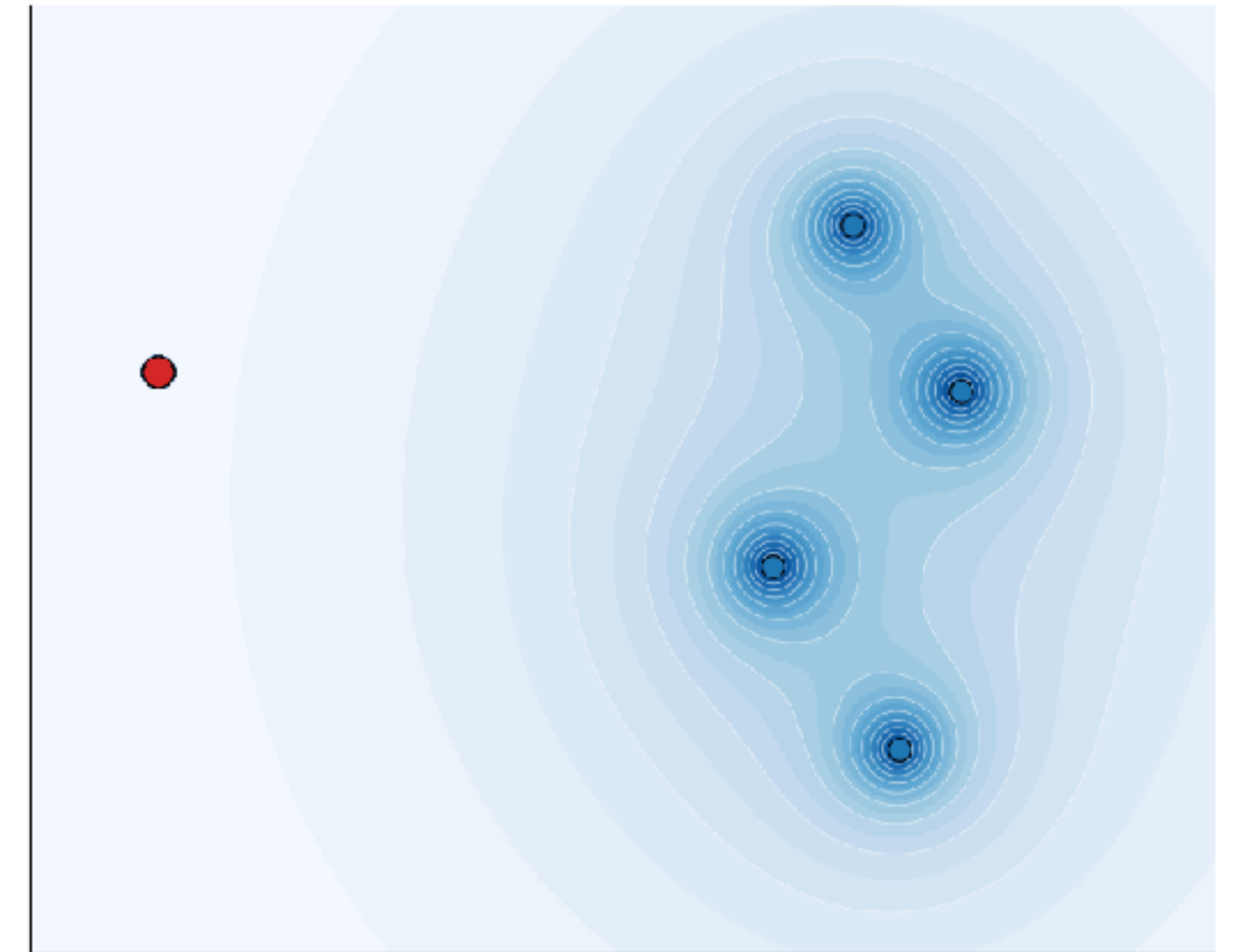
At every position ($\mathbf{x}$), the landscape assigns one number: $\phi(\mathbf{x})$.

$$\phi(\mathbf{x}) = \sum_i \frac{kq_i}{\sqrt{\|\mathbf{x} - \mathbf{r}_i\|^2}}$$

The red particle moves locally towards the lower potential.



Note that were the particle ends up depends on where it starts:



Section 1| A charged particle in a physical system: **The force vector field**

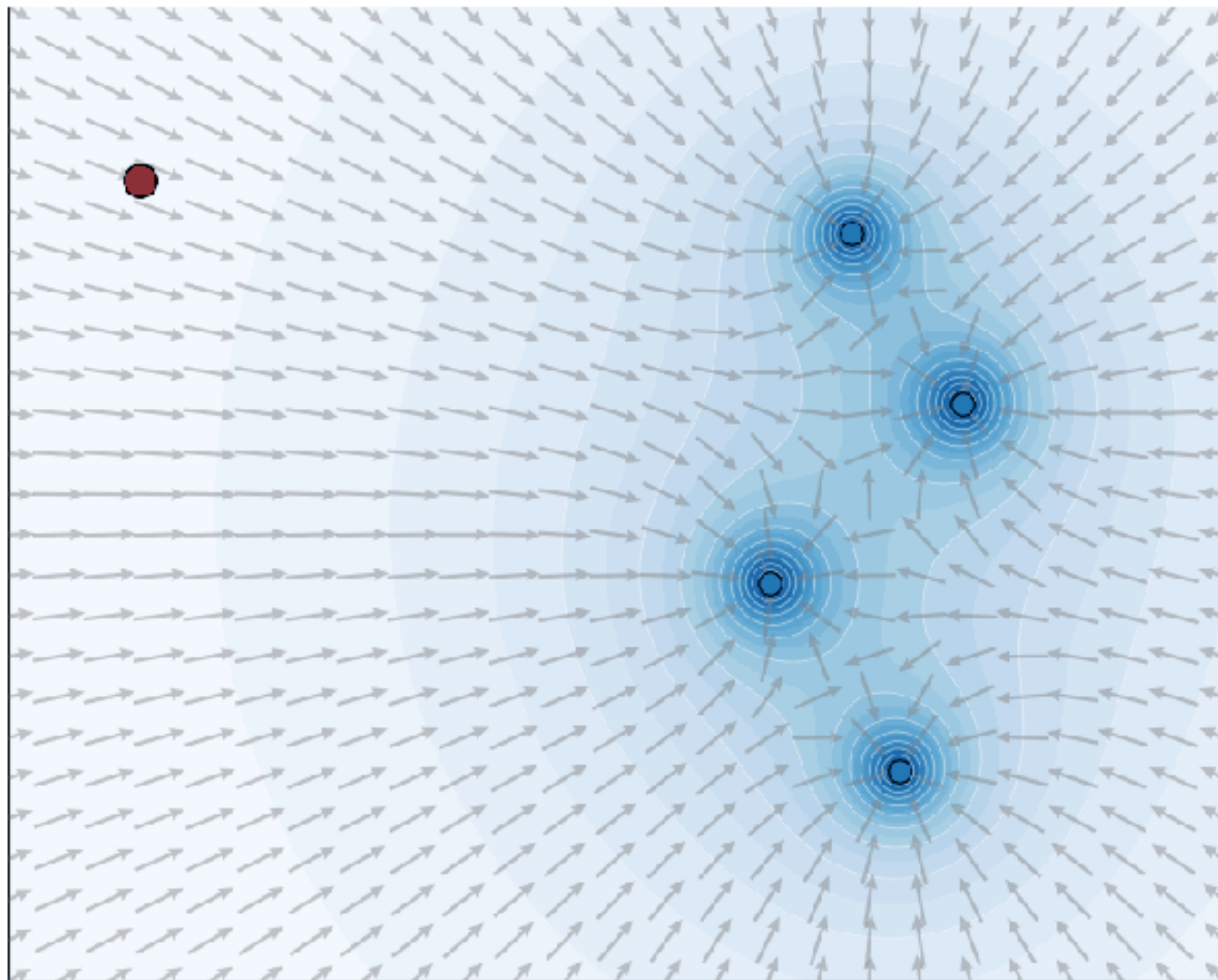
But how does the particle know where to go?

The potential $\phi(x)$ gives only a scalar value at each position.

To get a direction, we take its gradient: $\phi(x) \in \mathbb{R} \implies \nabla \phi(x) \in \mathbb{R}^d$

The gradient turns the scalar potential into a vector field, and for a positive charge the force is: $q = 1$:

$$F(x) = - \nabla \phi(x)$$



Arrows show: $F(x) = - \nabla \phi(x)$

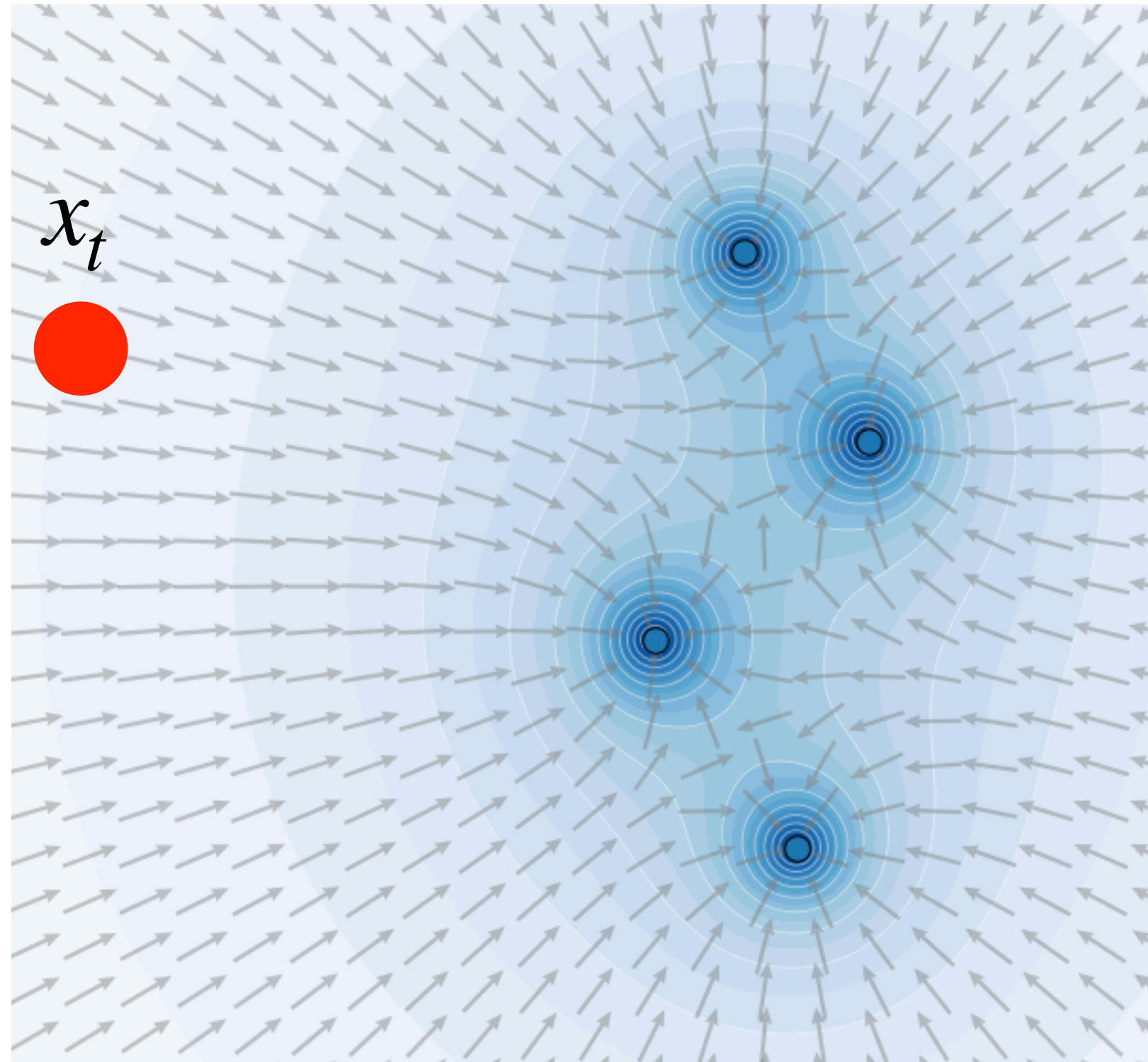
Section 1| A charged particle in a physical system: **The motion in time as small steps**

The force field tells the particle where to move from its current position. So we update the particle in small steps:

$$x_{t+\Delta t} = x_t + \Delta t F(x_t)$$

Or:

$$x_{t+\Delta t} = x_t + (-\Delta t \nabla \phi(x_t))$$



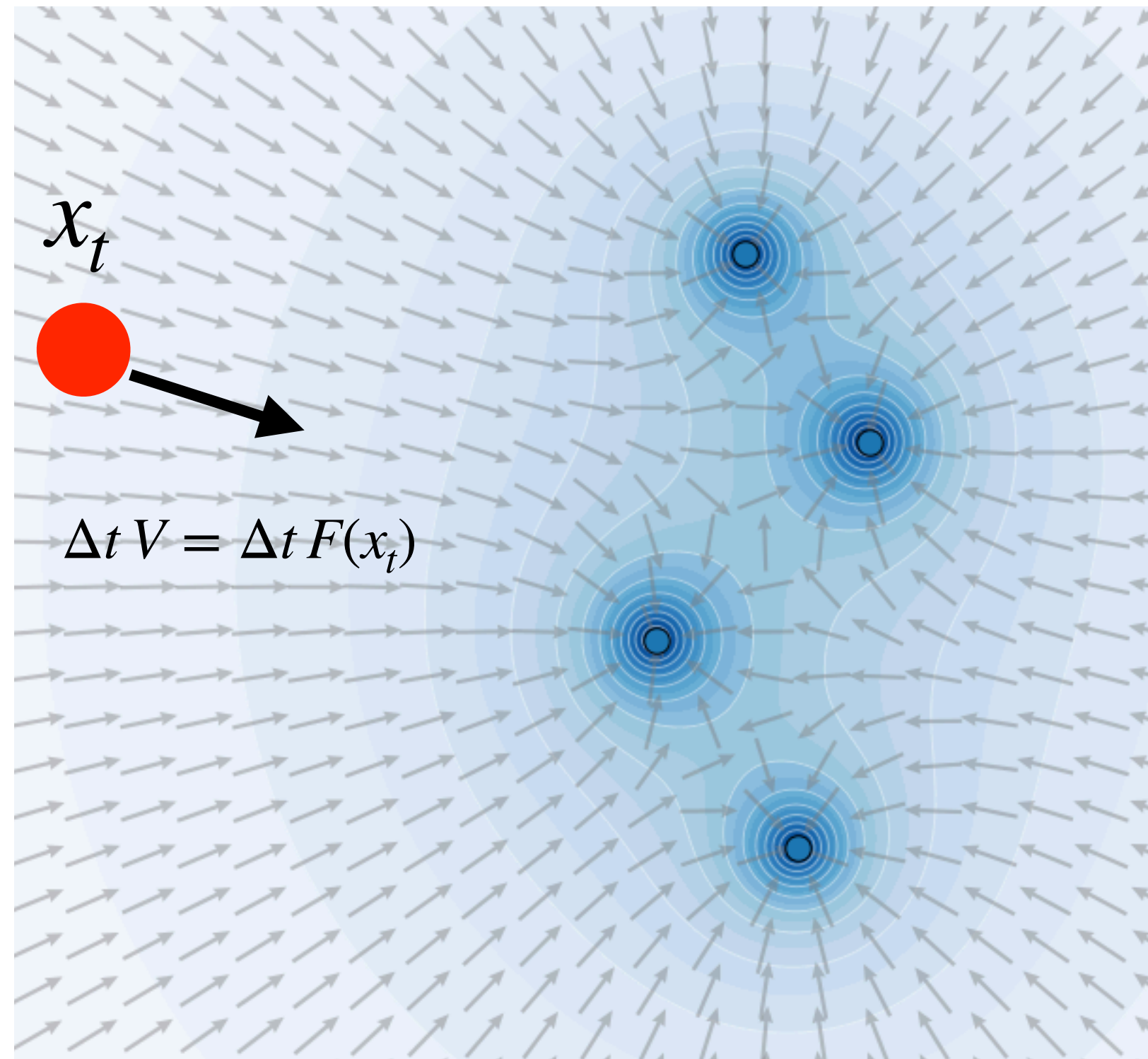
Section 1| A charged particle in a physical system: **The motion in time as small steps**

The force field tells the particle where to move from its current position. So we update the particle in small steps:

$$x_{t+\Delta t} = x_t + \Delta t F(x_t)$$

Or:

$$x_{t+\Delta t} = x_t + (-\Delta t \nabla \phi(x_t))$$



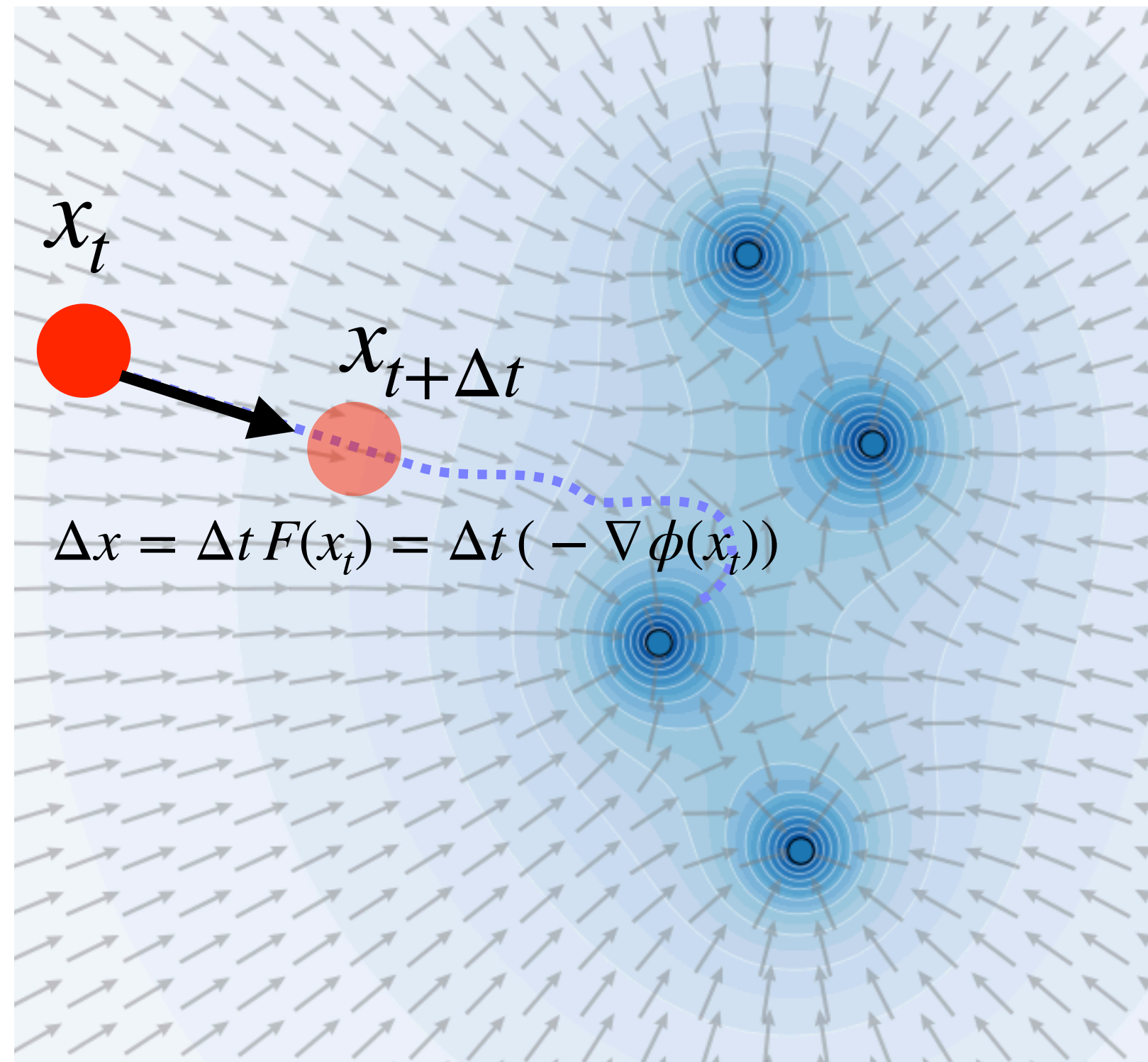
Section 1| A charged particle in a physical system: **The motion as small steps**

The force field tells the particle where to move from its current position. So we update the particle in small steps:

$$x_{t+\Delta t} = x_t + \Delta t F(x_t)$$

Or:

$$x_{t+\Delta t} = x_t + (-\Delta t \nabla \phi(x_t))$$



Also note that:

$$x_{t+\Delta t} - x_t = \Delta t F(x_t)$$

$$x_{t+\Delta t} - x_t = \Delta t F(x_t)$$

$$\frac{x_{t+\Delta t} - x_t}{\Delta t} = F(x_t)$$

Section 1| A charged particle in a physical system: From small steps to an ODE

If in this equation:

$$\frac{x_{t+\Delta t} - x_t}{\Delta t} = F(x_t)$$

We take the limit $\Delta t \rightarrow 0$ we get:

$$\frac{dx}{dt} = F(x) = (-\nabla \phi(x))$$

This is a first order ordinary differential equation (ODE)



To solve this ODE numerically we use Euler method, which anyways, uses the discretized version:

$$x_{n=0} = x_0$$

$$x_{n+1} = x_n + \Delta t F(x_n) \quad \text{Or} \quad x_{n+1} = x_n + \Delta t (-\nabla \phi(x_n))$$

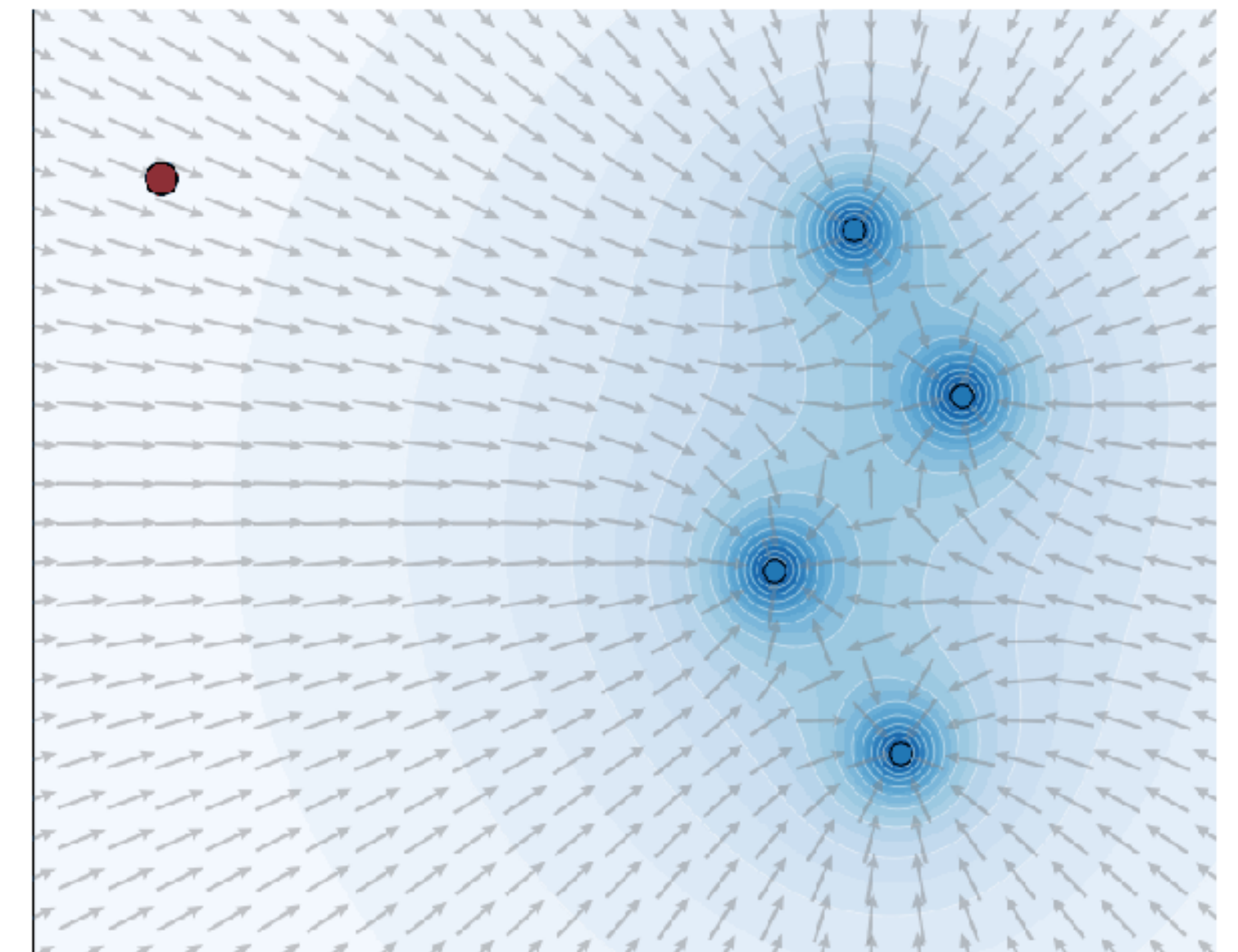
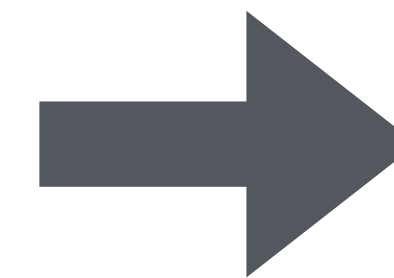
Pseudo-Code:

Start with x_0

For $n = 0, 1, 2, \dots$

compute force: $F(x_n) = -\nabla \phi(x_n)$

update: $x_{n+1} = x_n + \Delta t F(x_n)$



Section 1| A charged particle in a physical system: Can we reverse the motion?

For deterministic motion, every step is controlled by the field:

In discrete time steps, forward: $x_{n+1} = x_n + \Delta t F(x_n)$

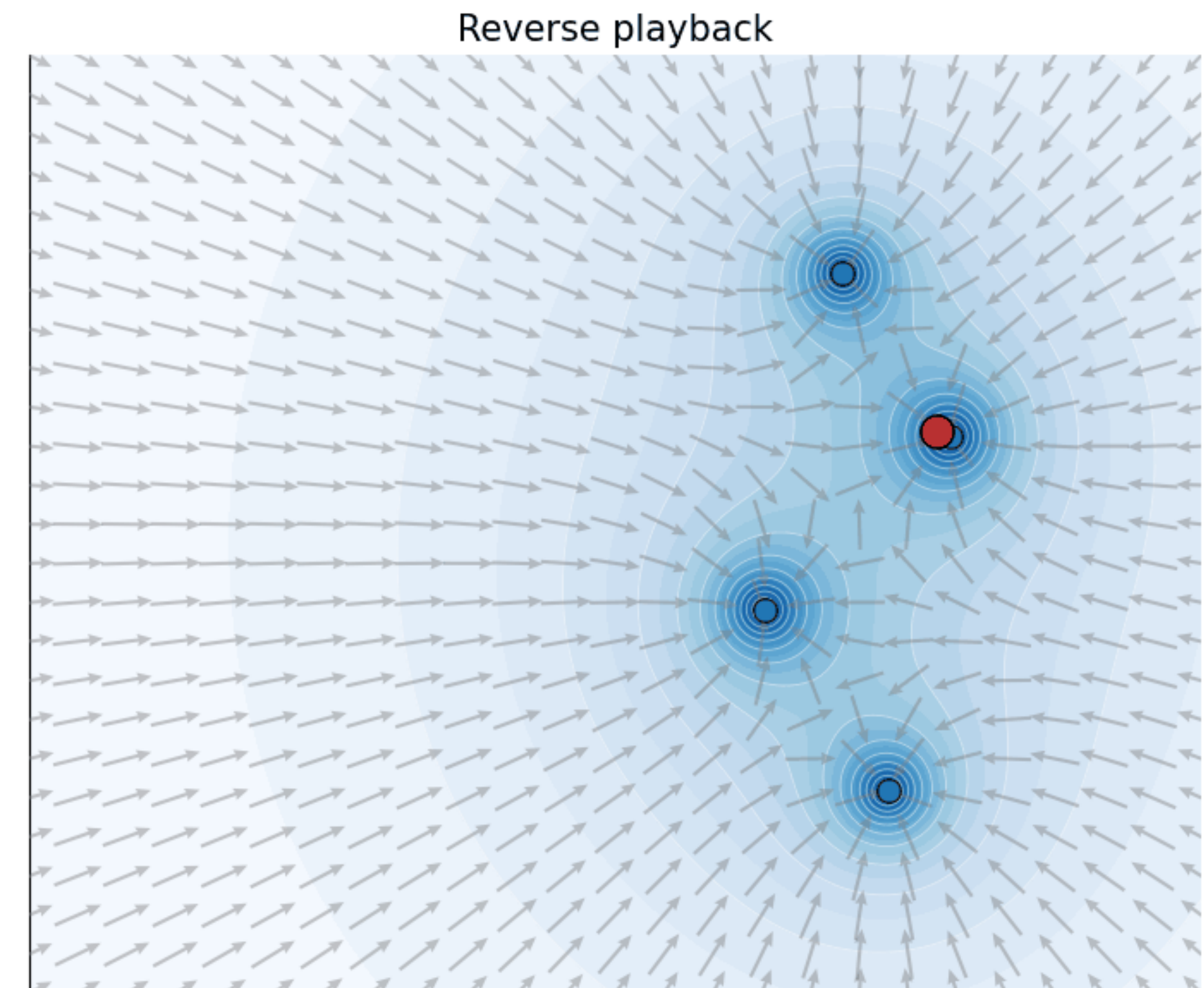
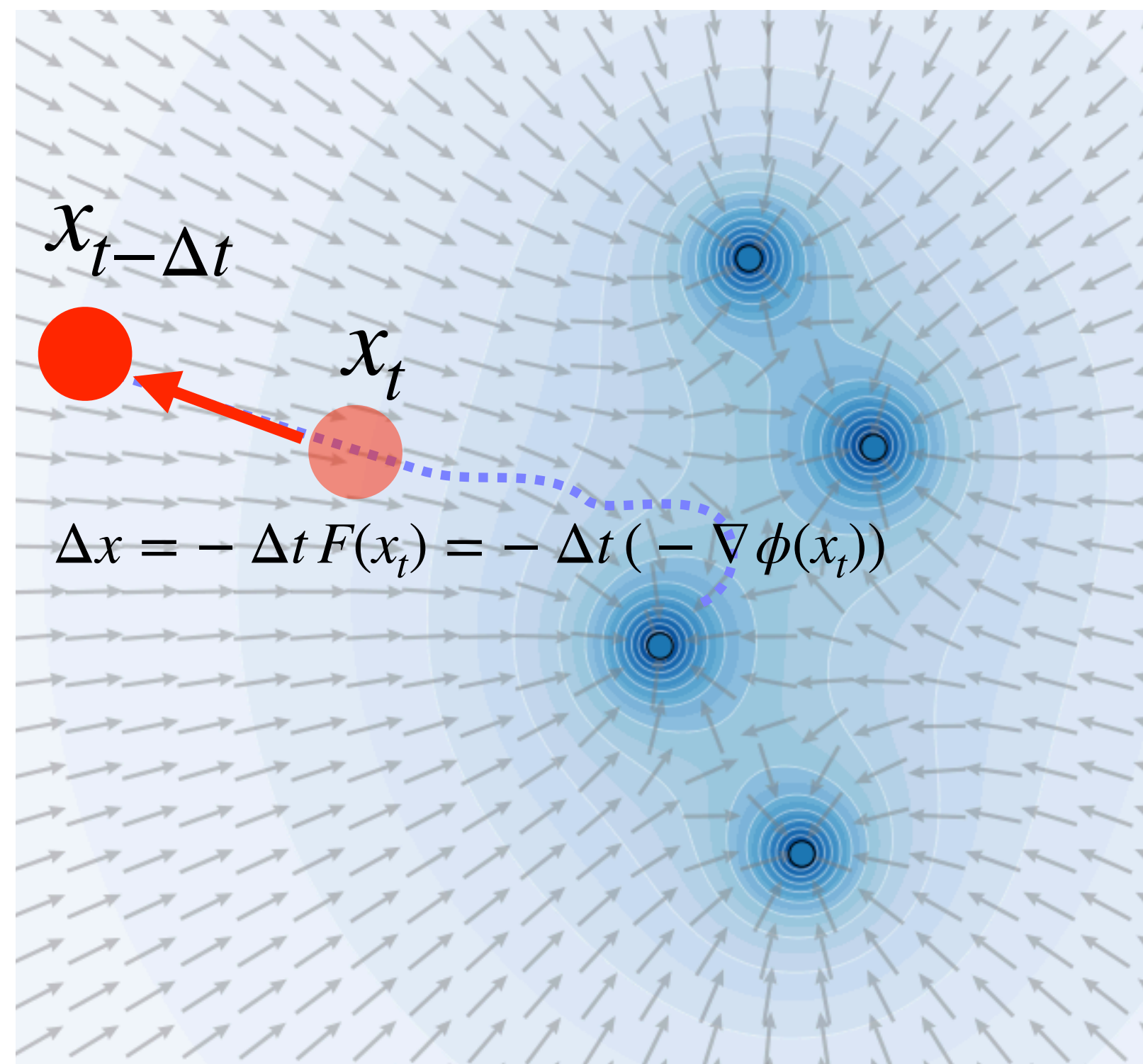
Backward: $x_{n-1} = x_n - \Delta t F(x_n)$

In continuous time, forward:

$$\frac{dx}{dt} = F(x)$$

Backward:

$$\frac{dx}{dt} = -F(x)$$



Section 1| A charged particle in a physical system: What if the particle also jitters?

So far, motion was deterministic: $x_{n+1} = x_n + \Delta t F(x_n)$

But real particles also receive random microscopic kicks (or noise from environment). Let's include them into the equation. The update becomes:

$$x_{n+1} = x_n + \Delta t F(x_n) + (\text{Random kick})$$

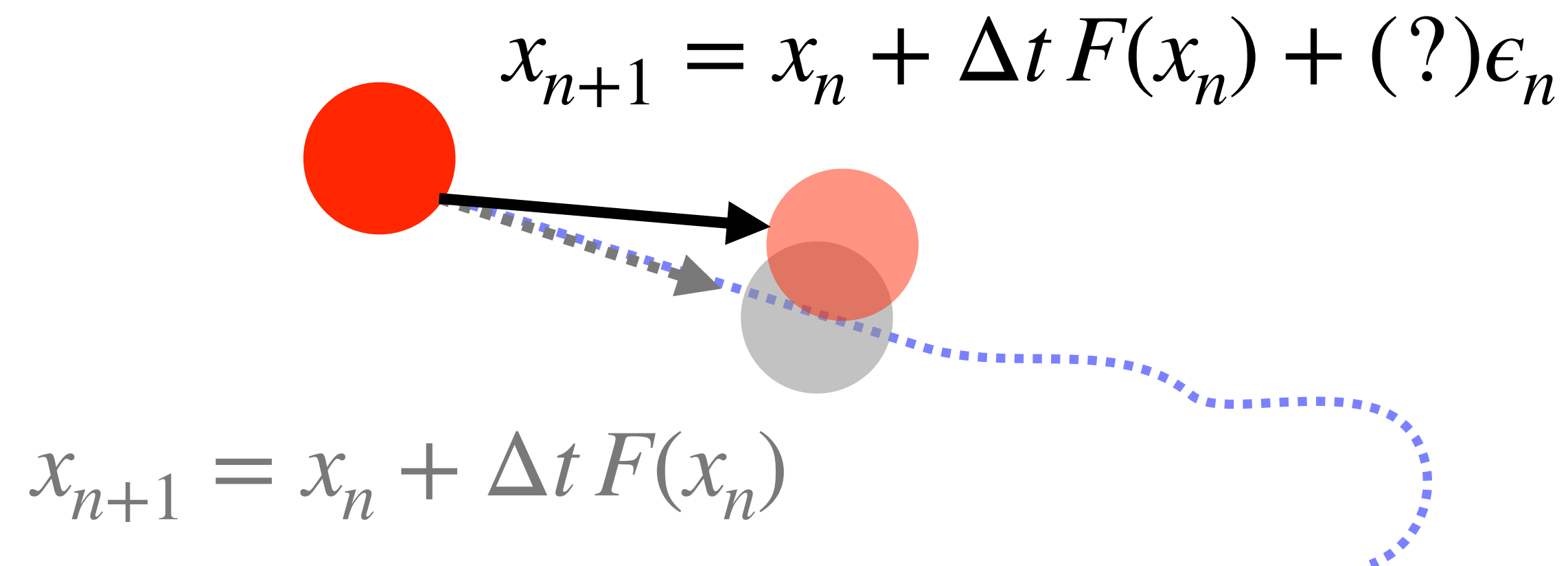
We model each *Random kick* as Gaussian noise:

$$\text{Random kick} = (?)\epsilon_n$$

$$\epsilon_n \sim \mathcal{N}(0, I)$$

ϵ_n is a random vector

The scalar (?) is a factor that controls the size of the random kick in incremental step Δt .

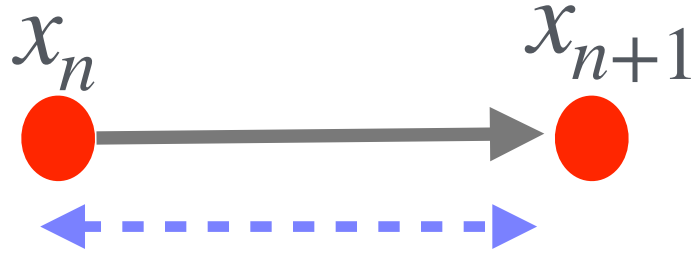


Section 1| A charged particle in a physical system: How the noise increases in step Δt

To understand this, lets take a deeper look at this equation:

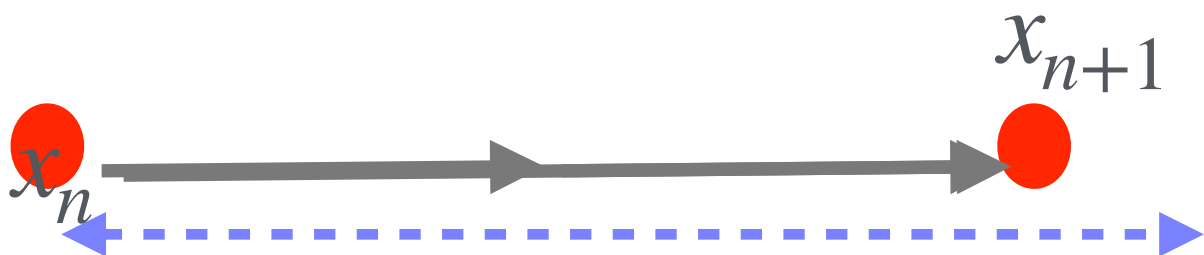
$$x_{n+1} = x_n + \Delta t F(x_n) + (?)\epsilon_n$$

Displacement due to Force is linear in (Δt) and it makes total sense:



Displacement= $\Delta t F(x_n)$

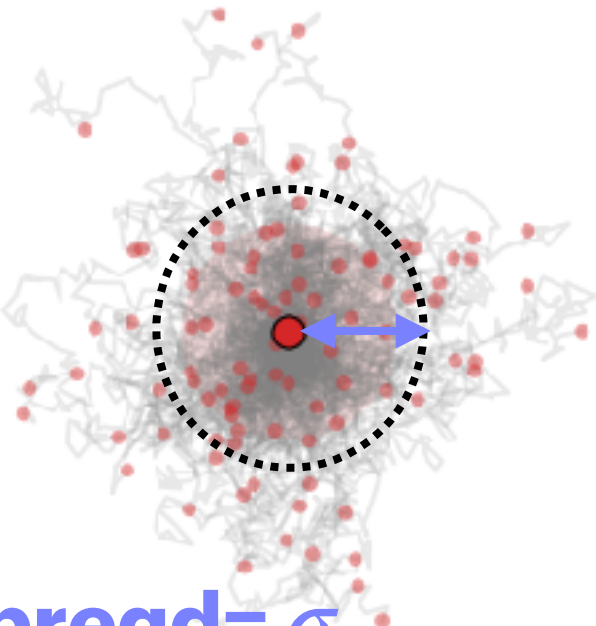
Lets say in another step $\Delta t' = 2\Delta t$:



Displacement= $\Delta t' F(x_n) = 2\Delta t F(x_n)$

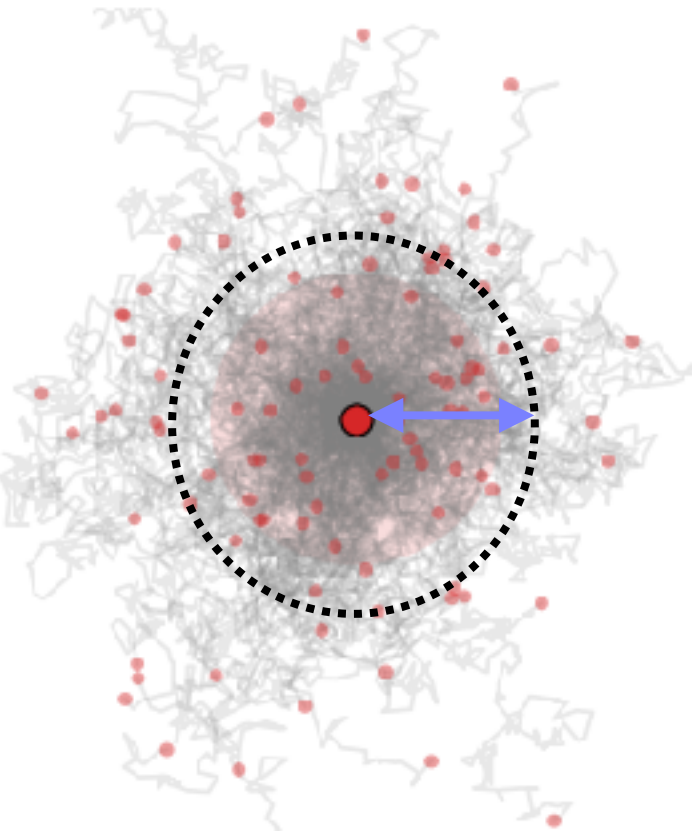
drift displacement due to Force is linear in time ($\propto \Delta t$)

Displacement due to random kicks in duration of Δt :



Average spread= σ

Displacement due to random kicks in duration of $\Delta t' = 2\Delta t$:



Average spread= $\sigma' = \sqrt{2}\sigma$

Average spread due noise term scales with $\propto \sqrt{\Delta t}$

Therefor the term $(?)$ should be proportional to $\sqrt{\Delta t}$

Section 1| A charged particle in a physical system: The noisy update function

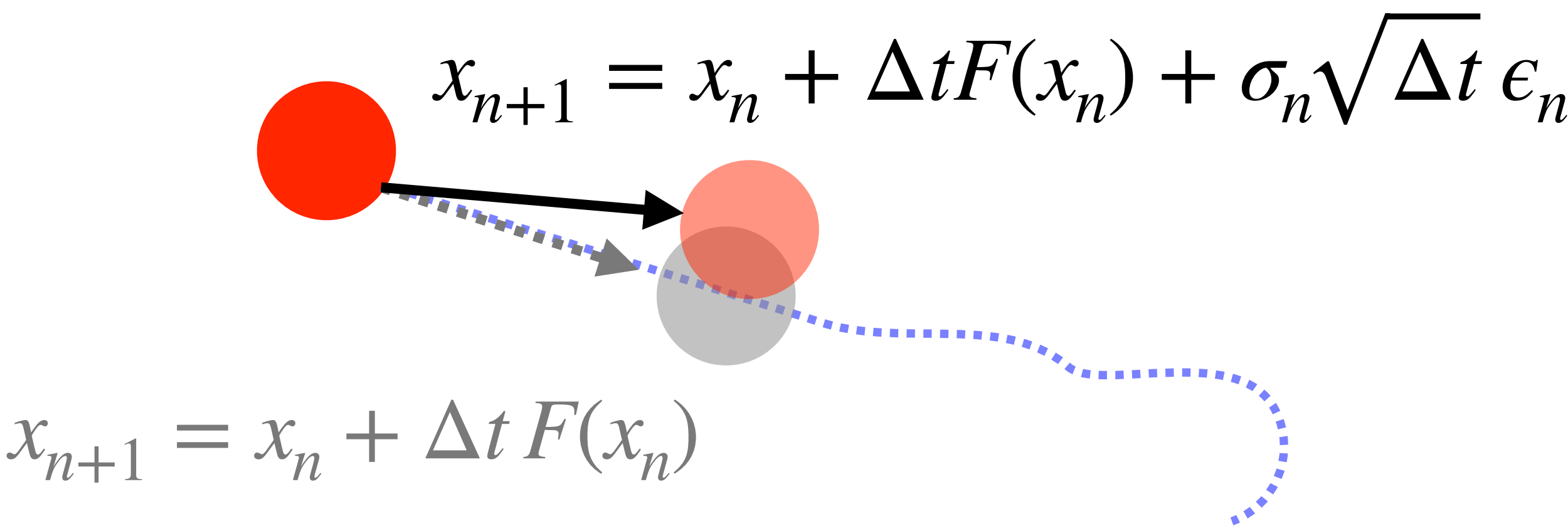
Given our previous slide:

drift displacement $\propto \Delta t$
noise spread $\propto \sqrt{\Delta t}$

Therefore the update becomes:

$$x_{n+1} = x_n + \Delta t F(x_n) + \sigma_n \sqrt{\Delta t} \epsilon_n$$

where: $\epsilon_n \sim \mathcal{N}(0, I)$, $\epsilon_n \in \mathbb{R}^d$ is a random vector and $\sigma_n \in \mathbb{R}$ is a scalar controlling the noise strength at step n .



Section 1| A charged particle in a physical system: From noisy update to an SDE (Stochastic Differential Equations)

We had:

$$x_{n+1} - x_n = \Delta t F(x_n) + \sigma_n \sqrt{\Delta t} \epsilon_n$$

Let's define noise as: $\Delta W_n = \sqrt{\Delta t} \epsilon_n$. This is just a definition, to simplify our equations, and means random displacement produced during the time interval Δt .

$$x_{n+1} - x_n = F(x_n) \Delta t + \sigma_n \Delta W_n$$

For small Δt ($\Delta t \rightarrow 0$) we can write:

$$dx = F(x, t) dt + \sigma(t) dW_t$$

$$\langle \Delta W_n^2 \rangle = \Delta t \implies \langle dW^2 \rangle = dt$$

The numerical method for an SDE is basically the noisy version of Euler:

$$x_{n+1} = x_n + F(x_n, t_n) \Delta t + \sigma(t_n) \sqrt{\Delta t} \epsilon_n$$
$$\epsilon_n \sim \mathcal{N}(0, I_d)$$

Euler-Maruyama Pseudo code:

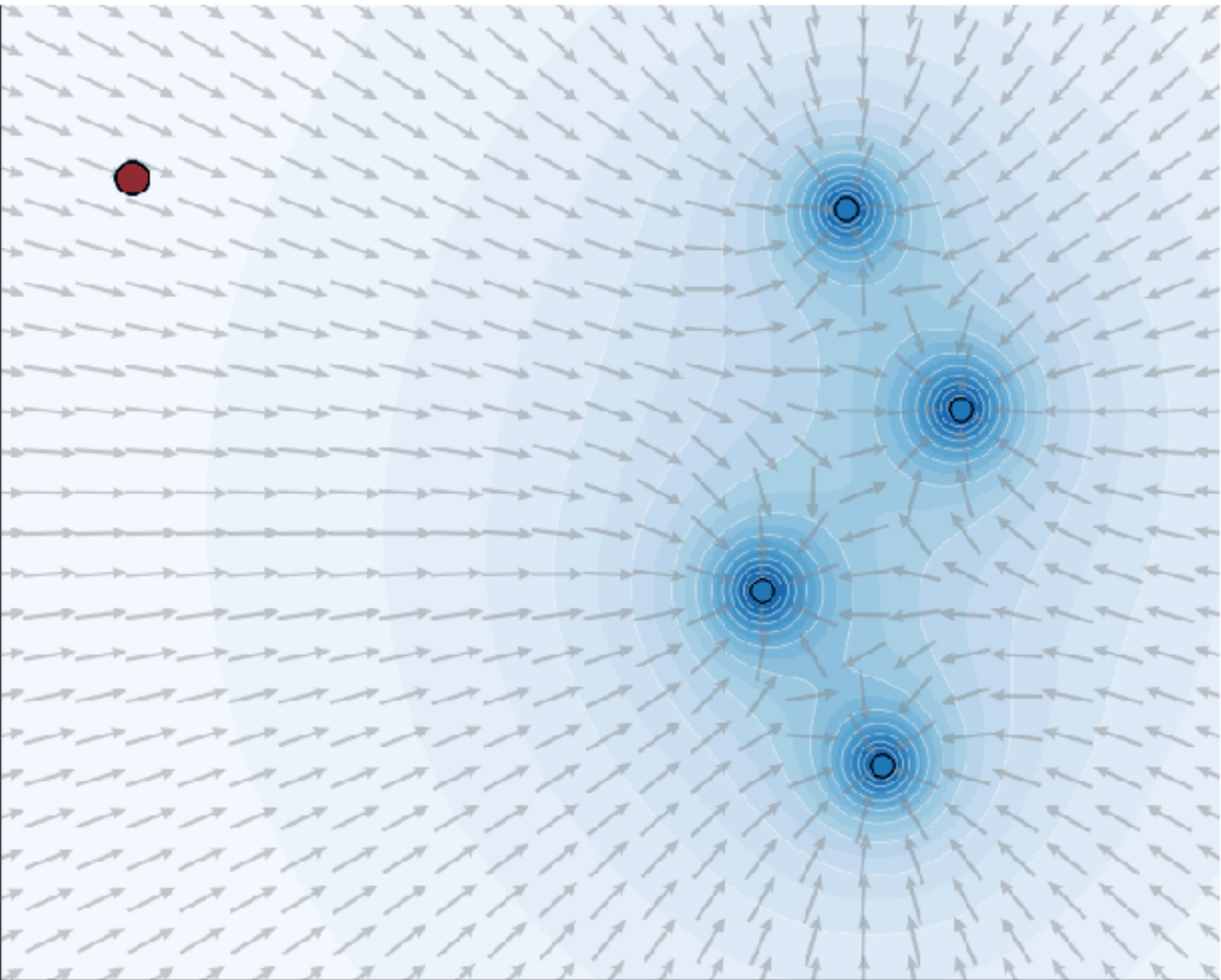
Start with $x(n=0)$

For $n = 0, 1, 2, \dots$

sample noise: $\epsilon_n \sim \mathcal{N}(0, I_d)$

compute drift: $F(x(n))$

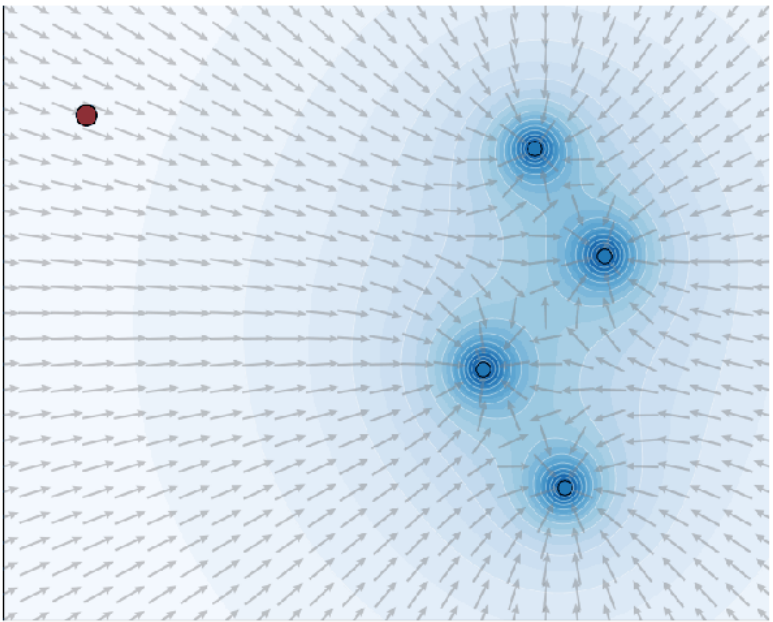
update: $x(n+1) = x_n + \Delta t F(x_n) + \sigma_n \sqrt{\Delta t} \epsilon_n$



Section 1| A charged particle in a physical system: **SDE trajectories as evolving densities**

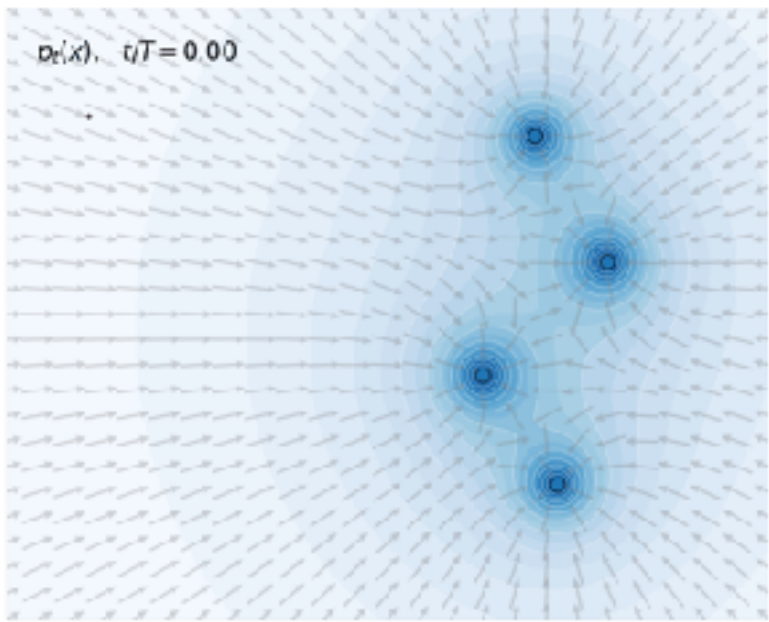
With an **ODE**, the same starting point gives the same path:

$$x_0 \longrightarrow x_t$$



With an **SDE**, the same starting point can give many different paths:

$$\begin{aligned} &\longrightarrow x_t^{(1)} \\ &\longrightarrow x_t^{(2)} \\ &\dots \\ &\longrightarrow x_t^{(N_{samples})} \end{aligned}$$

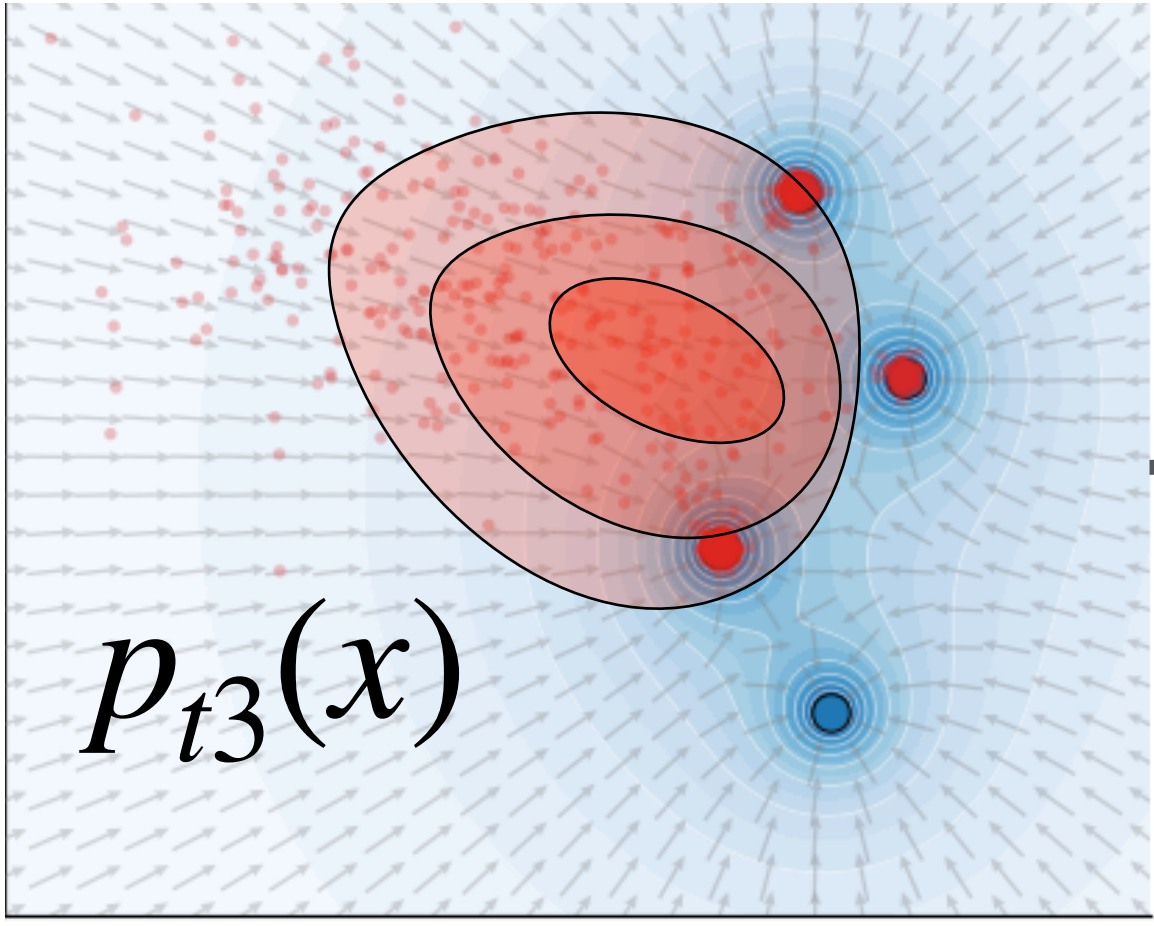
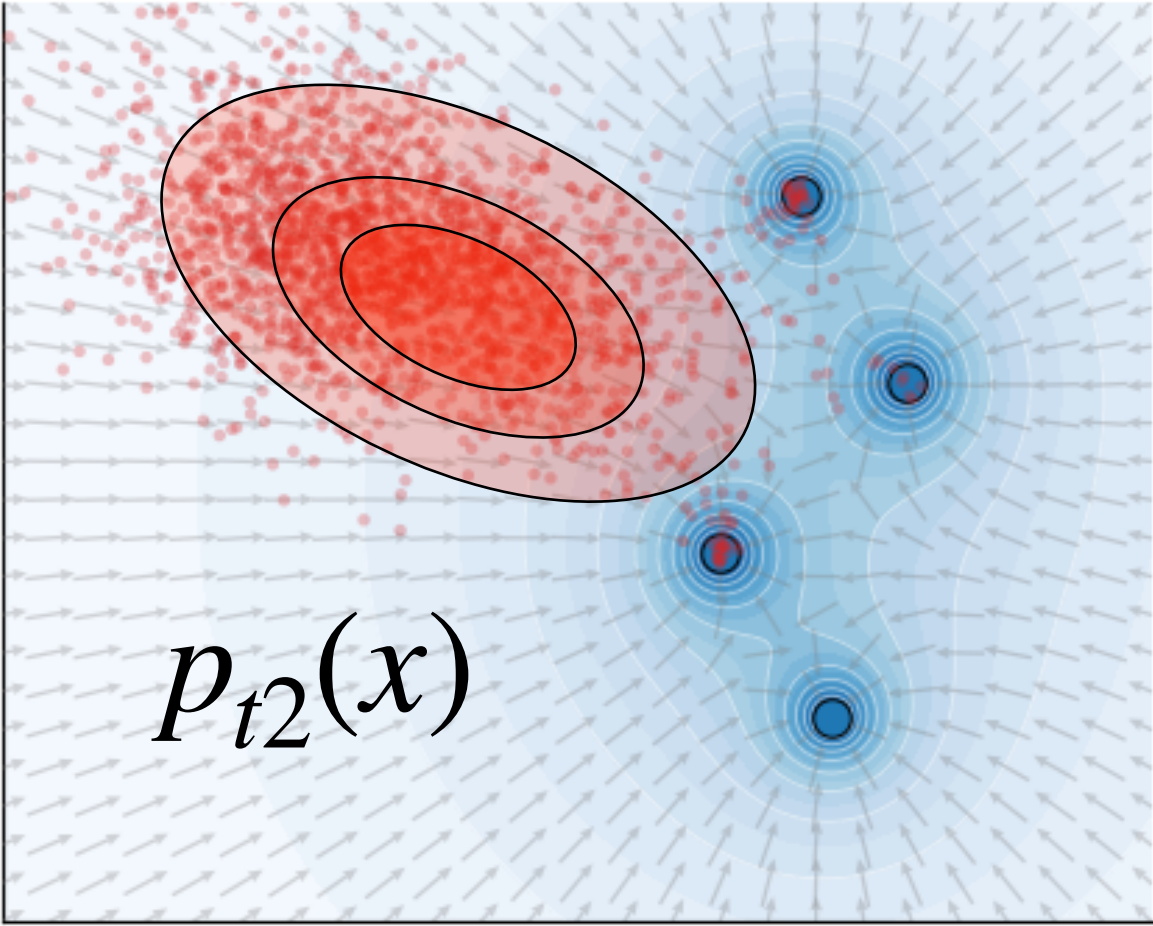
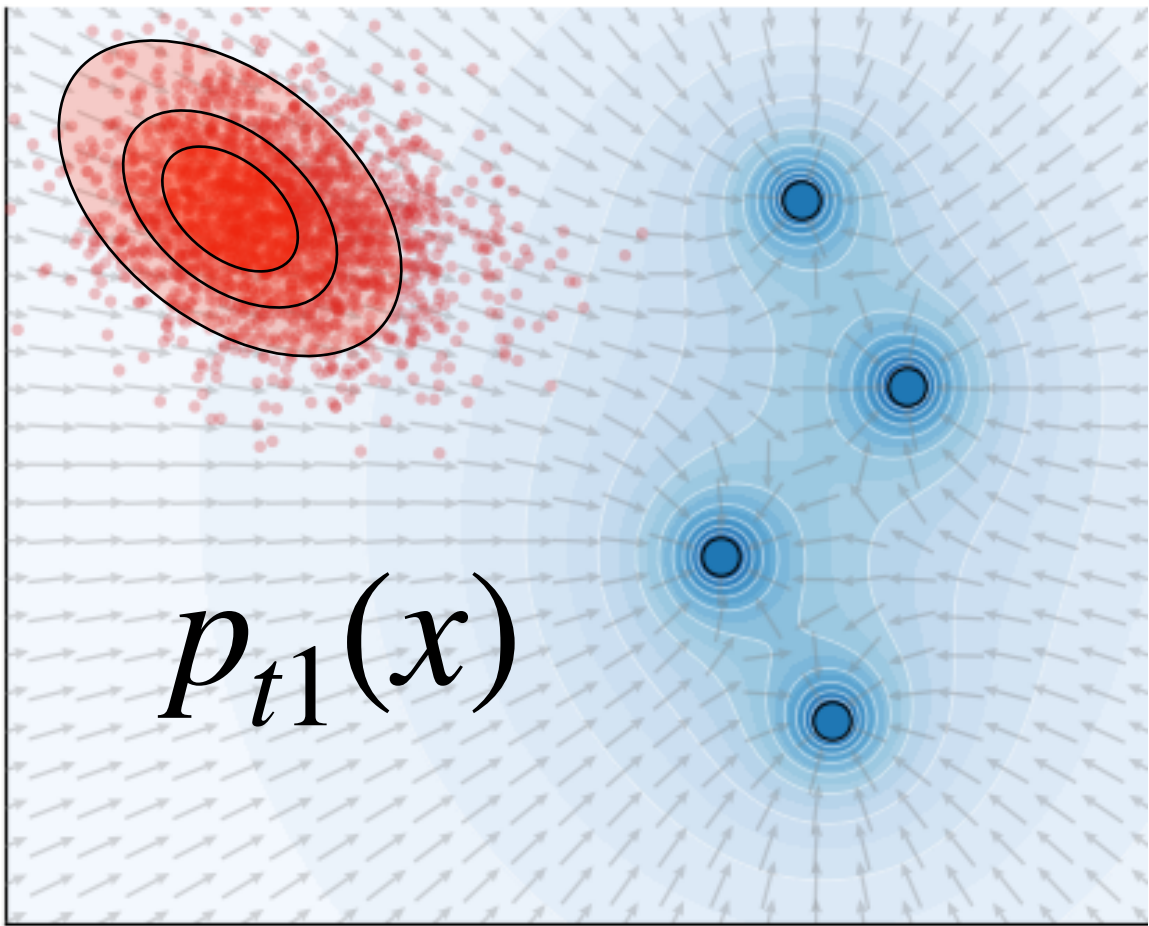


In an **SDE**, alternatively, instead of asking where one noisy particle goes, we can ask where many possible particles are likely to be.

So at time t , position is no longer a single point, it is a random variable described by a density:

$$x_t \sim p_t(x)$$

And density $p_t(x)$ evolves with time:



Section 1| A charged particle in a physical system: Can we reverse the noisy motion by just flipping the force?

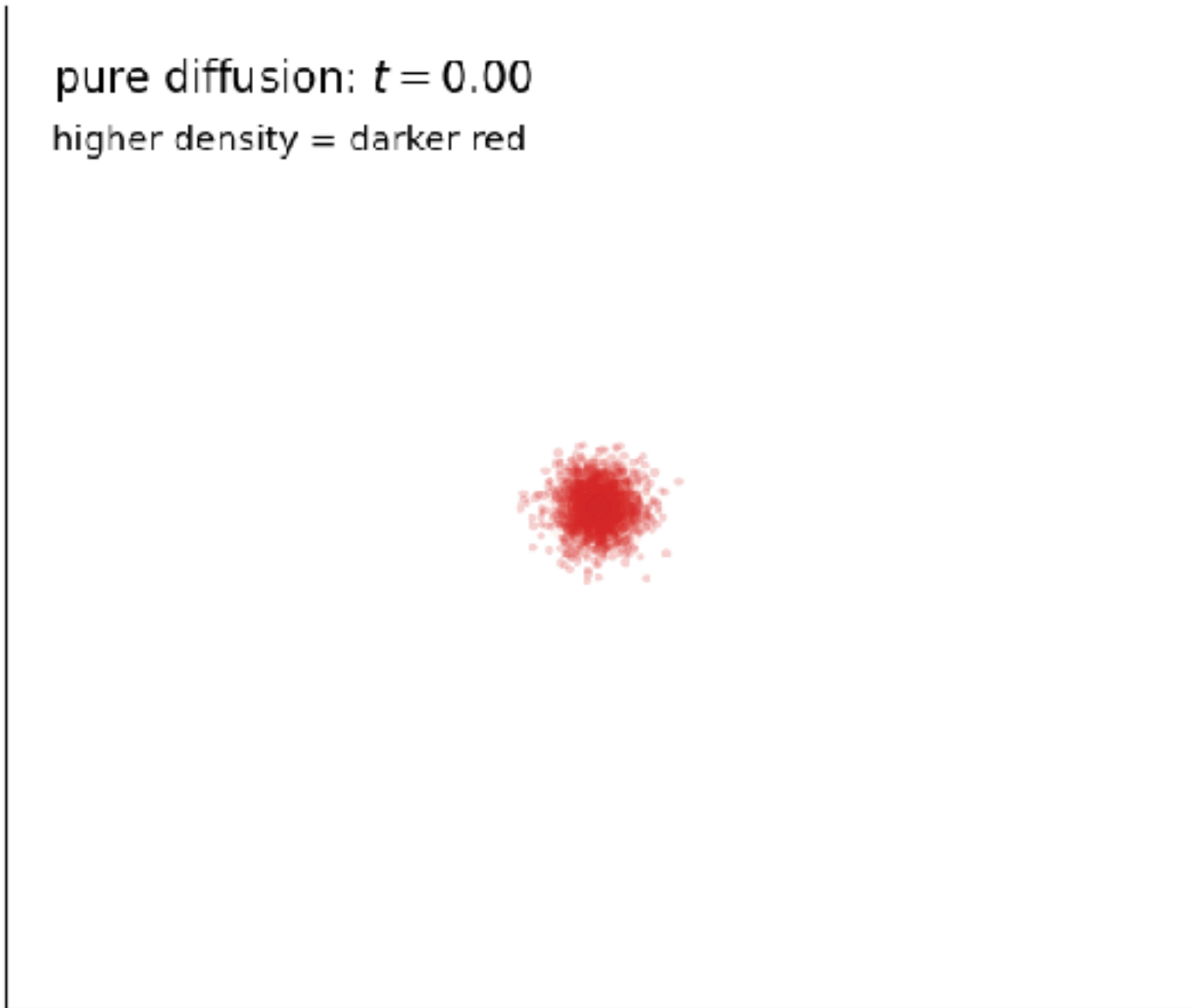
So for noisy motion with **SDE**, we might guess: $dx = -F(x) dt + \sigma dW$ **BUT is wrong!**

Why is the naive reverse wrong? To guess where a particle came from, two things matter:

1) Force Field, $F(x)$: This part is easy, we can just reverse it as $-F(x)$

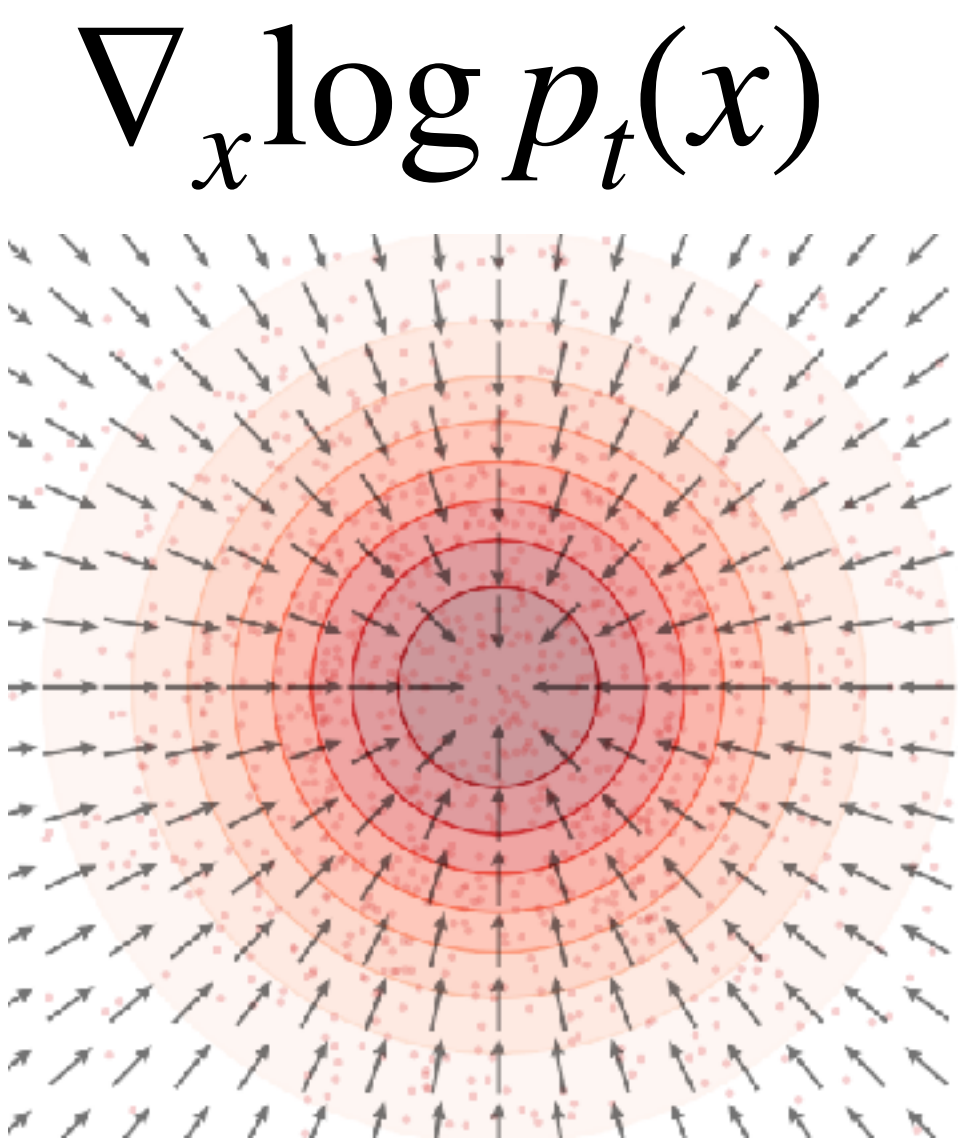
With Random kicks, or diffusionDiffusion make particles spread from high-density regions to low-density regions:

2) So when we reverse the process, likely previous positions are where there were more $p_t(x)$



How to find these regions?

We need to calculate $\nabla_x \log p_t(x)$
and see where it is pointing to



Section 1| A charged particle in a physical system: Reversing the SDE

Now we have all the ingredients, the forward noisy process is: $dx = F(x, t) dt + \sigma(t) dW_t$

and it creates a density at every time from many samples: $x_t \sim p_t(x)$

Anderson's proved the reverse process has two parts and is given by:

1) $-F(x, t)$ undoes the deterministic force

2) $\sigma(t)^2 \nabla_x \log p_t(x)$ undoes diffusion

$$dx = \left[-F(x, t) + \sigma(t)^2 \nabla_x \log p_t(x) \right] dt + \sigma(t) dW_t$$

REVERSE-TIME DIFFUSION EQUATION MODELS*

Brian D.O. ANDERSON**

Department of Electrical Engineering, The University of Newcastle, N.S.W. 2308, Australia

Received 13 November 1979

Revised 12 May 1980

Section 1| A charged particle in a physical system: Reversing the SDE

Reverse Euler-Maruyama

Start from the final forward cloud: $x_{\text{rev}}[0] \sim p_T(x)$

For $n = 0, 1, 2, \dots$

$t_n = T - n \Delta t$

sample noise: $\varepsilon_n \sim N(0, I_d)$

compute: $s = \nabla_x \log p_{t_n}(x_{\text{rev}}[n])$

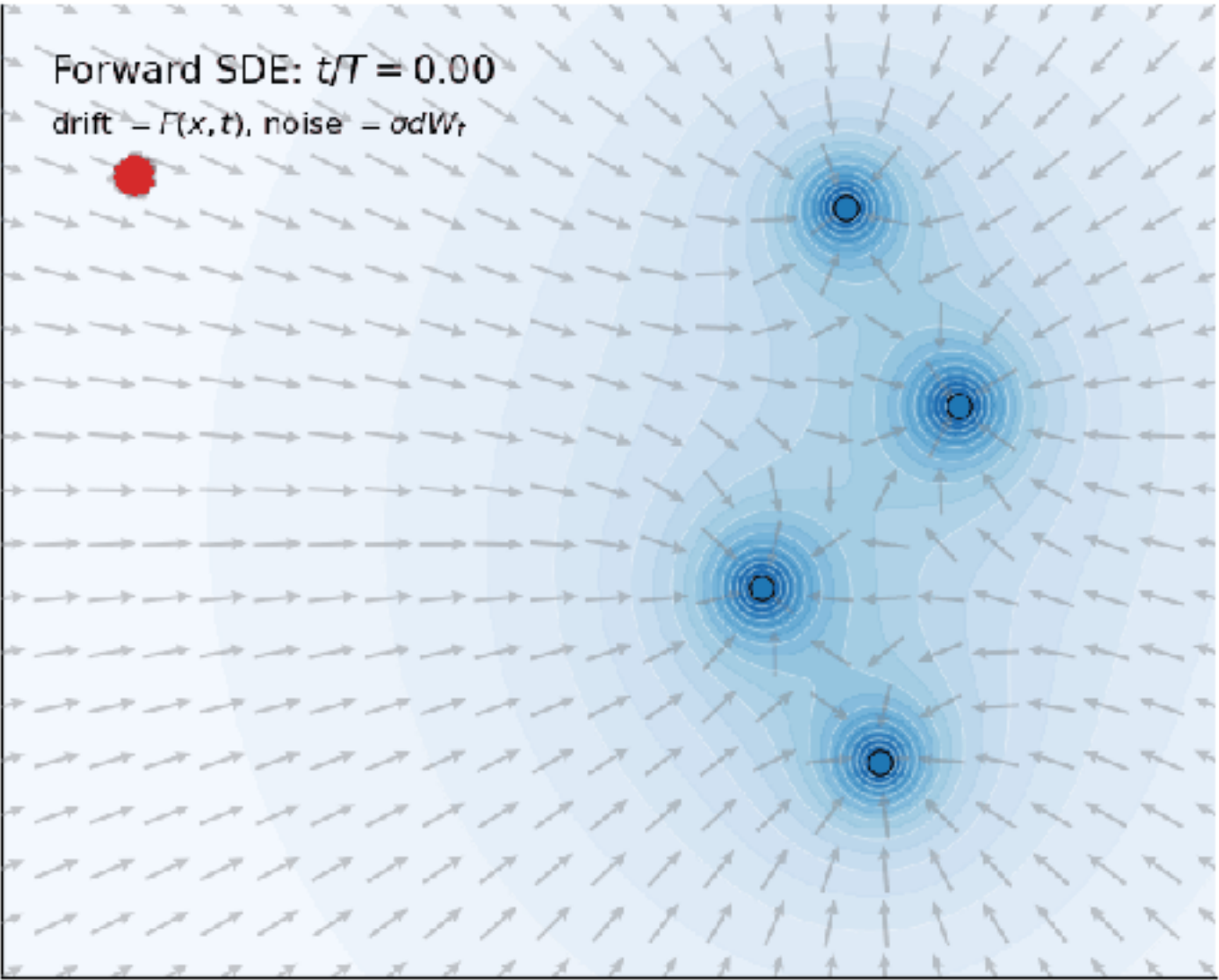
compute reverse drift: $b_{\text{rev}} = -F(x_{\text{rev}}[n], t_n) + \sigma(t_n)^2 s$

update: $x_{\text{rev}}[n+1] = x_{\text{rev}}[n] + \Delta t b_{\text{rev}} + \sigma(t_n) \sqrt{\Delta t} \varepsilon_n$

Note: In our toy code, we estimate $\nabla_x \log p_t(x)$ from the forward particle cloud.

This means we cannot reverse the SDE without knowing some information about its forward. What information we need? $\nabla_x \log p_t(x)$

$$dx = F(x, t) dt + \sigma(t) dW_t$$

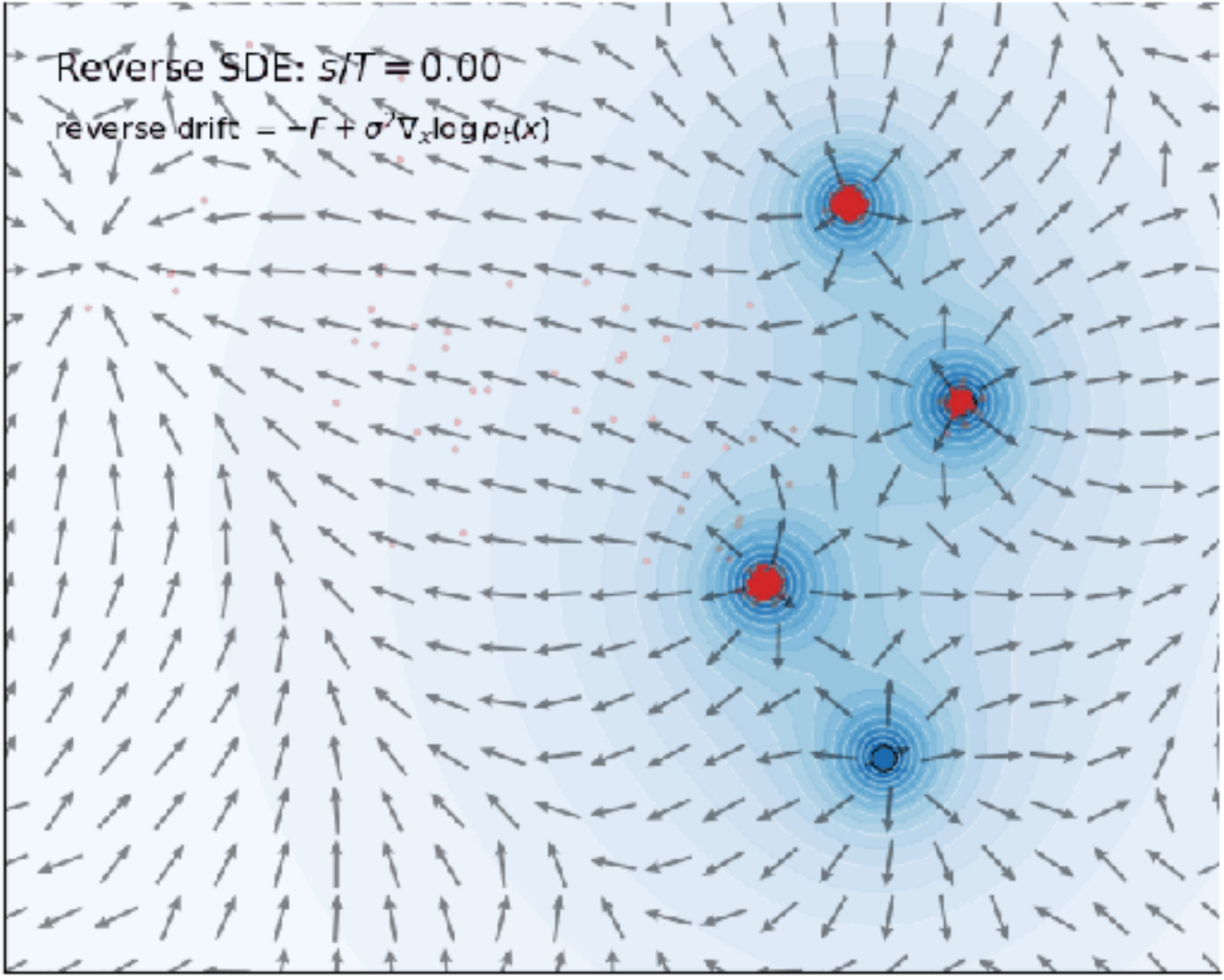


We estimate $\nabla_x \log p_t(x)$ by observing the forward path

Then can approximately reverse SDE:



$$dx = \left[-F(x, t) + \sigma(t)^2 \nabla_x \log p_t(x) \right] dt + \sigma(t) dW_t$$



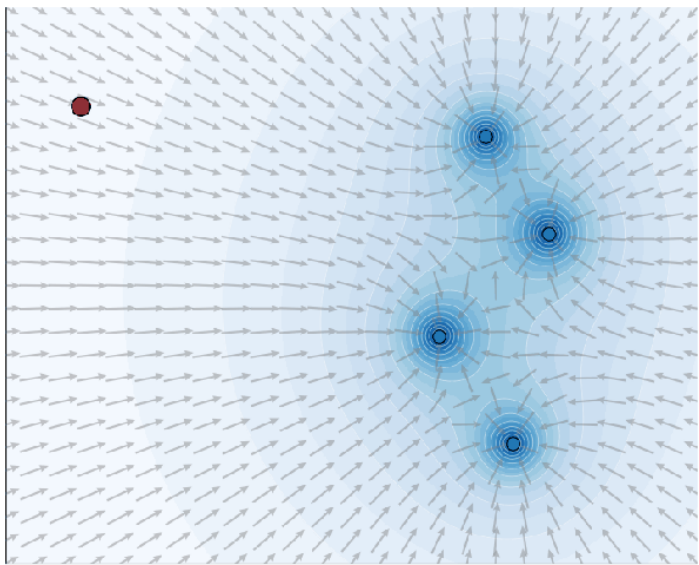
Summary of What we built in this lecture

We started from a simple physical picture: Particle moves due to a force $F(x)$

A) For deterministic motion, particles follow an **ODE**:

$$\frac{dx}{dt} = F(x, t)$$

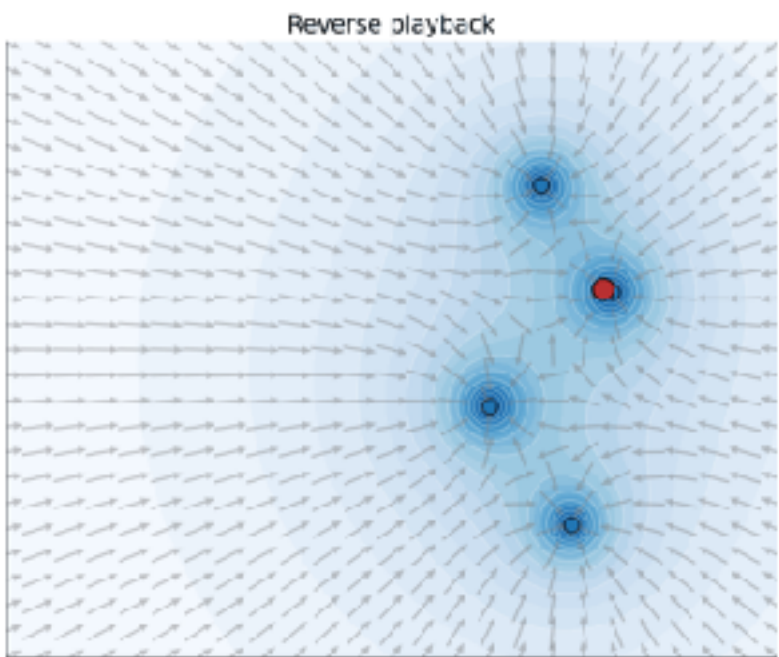
simulated by by Euler algorithm



B) And reversing **ODE** is easy:

$$\frac{dx}{dt} = -F(x, t)$$

simulated by by Euler algorithm



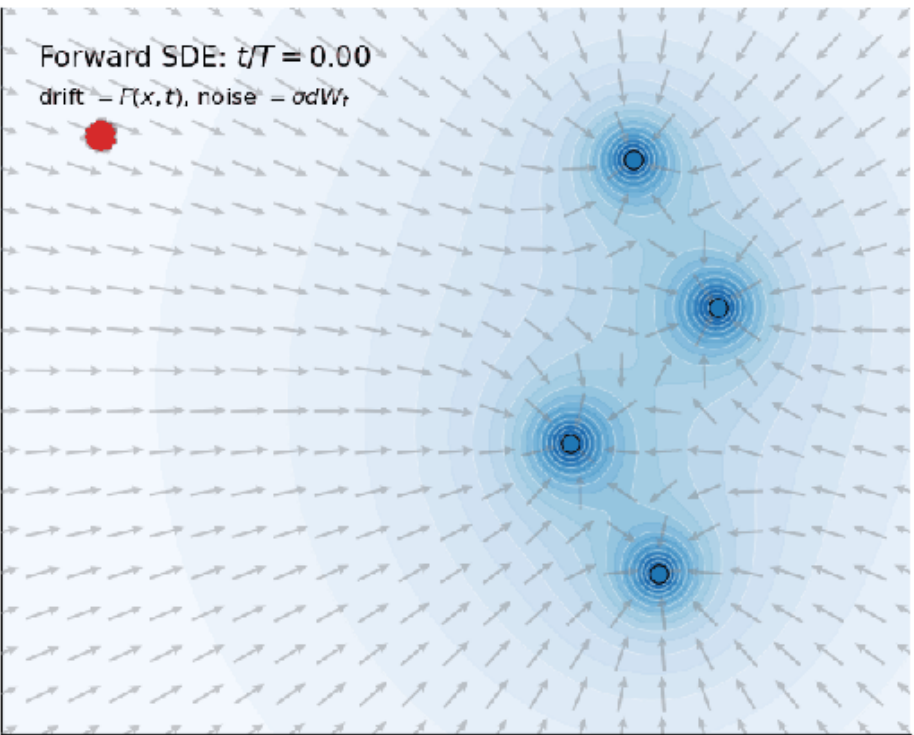
C) Then we added randomness and got an **SDE**:

$$dx = F(x, t) dt + \sigma(t) dW_t$$

Also we showed that

$$dW_t = \sqrt{dt} \epsilon \text{ and } \epsilon \sim \mathcal{N}(0,1)$$

simulated by by Euler-Maruyama algorithm



D) To reverse an **SDE**, flipping the force is not enough.

reverse-time SDE needs an extra density-correction

term: $\nabla_x \log p_t(x)$

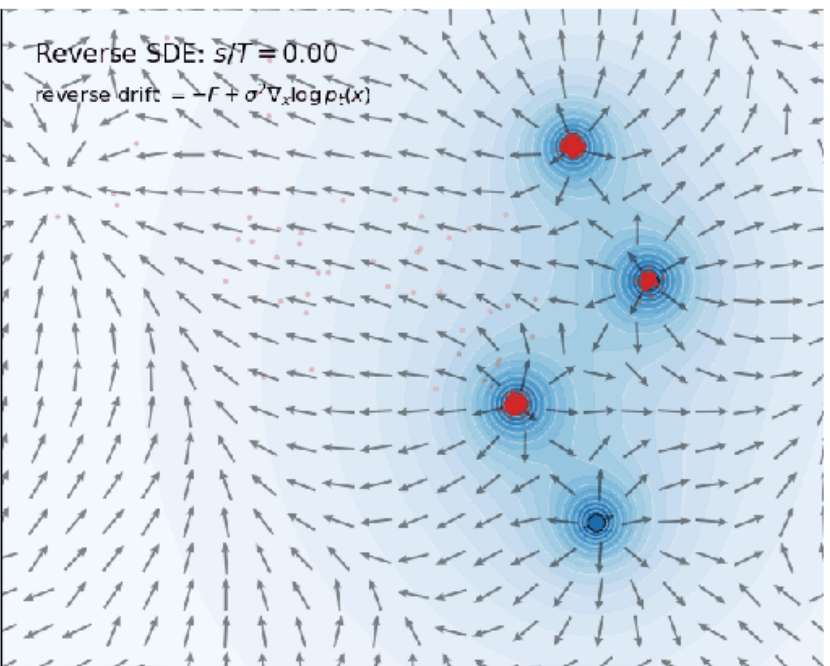
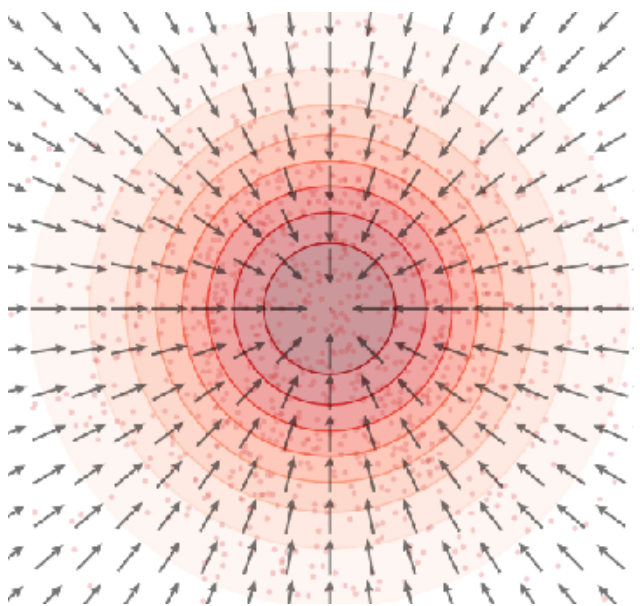
This time and location dependant term is not given to us,

We can maybe estimate it from forward process to be able to approximate reverse the **SDE**:

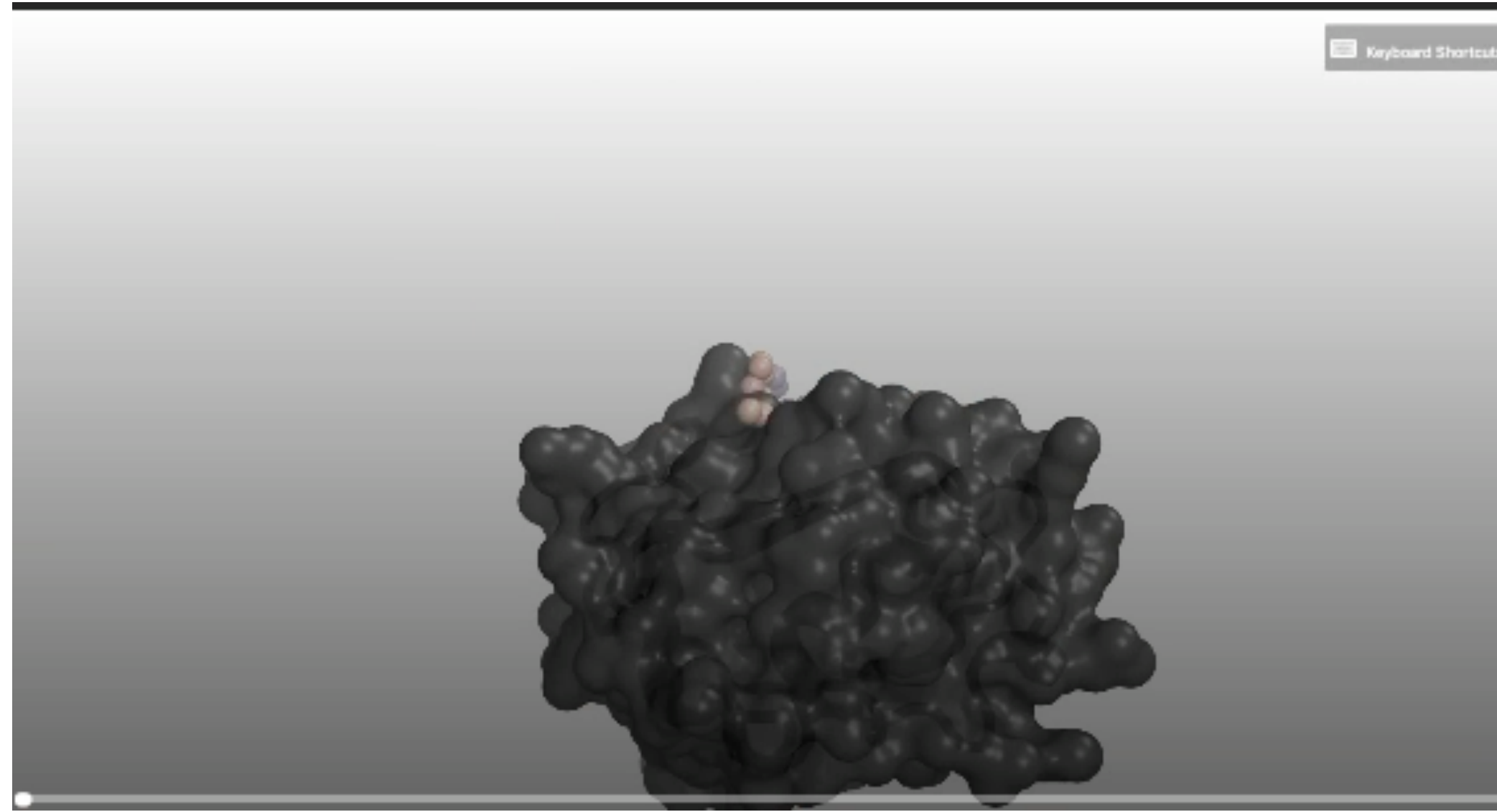
$$dx = \left[-F(x, t) + \sigma(t)^2 \nabla_x \log p_t(x) \right] dt + \sigma(t) d\bar{W}_t$$

simulated by Euler-Maruyama algorithm

$$\nabla_x \log p_t(x)$$



Why this lecture was related to modern Diffusion models?



In next lecture we will see that a big part of a diffusion model is to use a neural network $s_{\theta}(x, t)$ to learn, or estimate $\nabla_x \log p_t(x)$:

$$s_{\theta}(x, t) \approx \nabla_x \log p_t(x)$$